

BrawlBase

Martial Arts Search Platform

A platform that connects martial artists with sports professionals.

Name: Shayaan Ahmed

Candidate Number: 1204

Center Number: 51425



Table of Contents

Analysis.....	3
Success Criteria.....	12
Design.....	25
Technical Solution.....	57
Testing.....	87
Evaluation.....	131

Analysis

Introduction and Context

Over recent years, digital platforms have become increasingly embedded in the way individuals manage their health, fitness, and professional development.

Smartphones and web-based applications are now commonly used to track physical activity, analyse performance, and connect users with wider communities that share similar interests. This growth has been particularly evident in the fitness industry, where applications allow users to record workouts, monitor progress, and set performance goals.

While many of these platforms are effective for general fitness activities such as running, cycling, and gym-based training, they are far less suited to specialist sports. Martial arts, for instance, put significant emphasis on partnerships and training for skill matching and access to coaches and supporting professionals. This is despite growing technological advancements in fitness through digital programs. Currently, there are few systems that meet the requirements of martial artists.

The project outlines a proposal for creating a web-based system that can connect martial artists with proper training partners, gyms, coaches, and doctors. A centralized system can make martial artists progress, can improve their training, and can ensure consistency in their training across the martial arts field.

Problem Definition Martial artists

Especially those at an intermediate to advanced level, may face a significant level of hardship when attempting to locate a training partner for their combat style. Career success in combat sports depends to a great extent upon training with someone appropriate, i.e., at one step above in terms of ability and training, while fighting someone at one step or more below in terms of ability will definitely inhibit one's progress in combat sports.

Apart from that, martial artists also have a requirement for specific services like gyms, coaches, and medical professionals like physiotherapists. Currently, these services are also present on various platforms, where the user has to look through different websites or applications to meet his/her requirements.

Problem statement:

Experienced martial artists are unable to efficiently find training partners and support services that match their skill level and geographical location using existing digital platforms.

Why the Problem Matters

Lack of training partners and training services may have a direct impact on the training of the martial artist. Training with inappropriate training partners may result in plateaus, reduced motivation, and injuries. For those who wish to compete and make a living as professional martial artists, such a limitation may significantly impede their training and advancement as martial artists. Moreover, within the context of martial arts, there is a higher chance of injury. Delayed access to the relevant medical support means there are various long-term physical problems that may come about. Therefore, a platform that enables martial artists to search for relevant resources quickly is necessitated by its ability to reduce downtime.

Identifying the Major End User

The main end user for this system is Saad, a martial artist who works professionally in mixed martial arts. On a daily basis, Saad trains, records his sparring sessions, and views the replay to think about how he could improve. His commitment to training means that he requires consistent access to partners who can challenge him appropriately. Despite his commitment, Saad finds few training partners at his level in his locality. Many of the individuals available at nearby gyms are less experienced, limiting his ability to progress. Besides, in case of any injuries, finding skilled physiotherapists and coaches also takes a lot of time and effort, as no single platform guarantees access to all such services.

Needs of Primary End User

The main end user needs a system where he can. Find training partners who practice the same martial art and possess similar skill levels, find gyms and coaches that specialize in his sport of choice, access medical professionals with experience dealing with martial arts-related injuries. Instead, try to use one good service rather than multiple unrelated services. These needs will directly impact the requirements for the proposed system and validate the need to build a system for martial arts in particular.

Identification of secondary end users

Apart from the main end user, there are also several secondary users to be considered. This includes martial art coaches, gym owners, as well as medical practitioners, physiotherapists, and sports therapists. This group will be able to access a large number of clients or patients who are motivated aspiring athletes.

Stakeholders and Their Interests

The stakeholders who are primarily concerned with this project are the primary user, secondary users, and the system administrator. The primary user needs training connections, the secondary users need to offer their services, whereas the system administrator needs to ensure that the system provided offers accurate information, security, and reliability.

Review of Existing Solutions

Several existing platforms partially address aspects of this problem. Among them are Strava, Spartner, and Fightura. In each of these systems, part of the functionality is relevant, but none fully satisfy advanced martial artists' needs.

Strava Analysis

Strava is a popular fitness-tracking social network that allows people to track physical activities and share performance data. Its design is clean UI/UX, enabling social sharing-maybe effective in the tracking of endurance sports such as running and cycling. However, it does not offer skill-based matching of partners, nor is it designed with combat sport in mind; thus, it does not meet any of the identified core needs within this project.

Spartner Analysis

Spartner focuses more directly on combat sports by allowing users to search for gyms, fighters, and coaches. While that represents a step in the right direction, the platform currently lacks advanced filtering options, geographical flexibility, and community feedback in the form of reviews, which reduces the value for more advanced athletes.

Fightura Analysis

For physical attributes and experience level, Fightura provides very detailed filters. On the other hand, this platform has been facing issues with an outdated UI, limited regional coverage, and the absence of integrated messaging and service discovery, which restricts it as a comprehensive solution.

Analysis

Website	Strengths	Limitations
Strava (https://www.strava.com/)	▪ Clean Layout: The site has a straightforward design that makes browsing easy for users. ▪ Search Functionality: It includes a search bar and category filters, which help visitors quickly find specific partners or services. ▪ Descriptive Listings: Each listing provides basic contact information and a short description, making it convenient for users to browse relevant options.	▪ Feature Overload: The abundance of tools and data can overwhelm new users. ▪ Activity History Navigation: Finding past workouts can require excessive scrolling. ▪ Cluttered Social Feed: Hard to prioritise relevant content. Limited Filtering.
Spartner https://www.spartner.co.uk/	▪ Clean UI: Minimalistic and uncluttered design supports easy navigation. ▪ Good Search Features: Can filter for gyms, fighters and coaches. Even region/specific martial art searches ▪ Essential Listing Info: Contact details and short descriptions are readily available for quick browsing.	▪ No Interactive Map: Users can't visualise people's locations on a map. ▪ No Reviews or Ratings: Lack of community feedback reduces trust and engagement for fighters/gyms/coaches. ▪ Limited Filters: Can't filter by location radius, and there are only limited regions to search in, skill level, or availability and no way to save favourites.
Fightura https://www.fightura.com/	▪ Detailed Search Filters: Users can search by age, weight, height, stance, gender and experience level. ▪ Recent Activity Feed: Shows active users and recent matches, adding a social aspect/can create a related community.	• Outdated UI Design: Lacks visual polish and mobile responsiveness not as easy to navigate. ▪ No Map Integration: Users can't visualise sparring partners on a map or a specific area they want to look in. ▪ Limited Regions that the website supports:

	- Multi-Discipline Support: Caters to boxing, MMA and a variety of combat sports. - Quick Registration: Easy sign-up process for new users.	The areas that are stated in the filters are only specific to America. - No Messaging System: No built-in chat or contact feature for users - No Reviews or Ratings: Lack of community feedback reduces trust and engagement for fighters/gyms/coaches. - Limited User Dashboard: Lacks favourites, history, or profile setting management.
--	---	---

Comparison Of Languages

<i>Language</i>	<i>Type</i>	<i>Strengths</i>	<i>Weaknesses</i>	<i>Common Uses</i>
<i>Python</i>	High-level, interpreted	Easy to learn, readable syntax, a large library support and versatile. I have personal experience using Python, which has given me practical familiarity with it.	Slower execution speed compared to compiled languages, less suited for mobile apps.	Data science, machine learning, web development, automation, scripting
<i>Java</i>	High-level, compiled (to bytecode)	Platform independent, object-oriented, strong performance, widely used	Verbose syntax, slower startup time, and more complex than Python	Enterprise applications, Android apps, backend systems
<i>CSS</i>	Style sheet language	Controls design and layout, separates content from presentation, easy to learn.	Limited to styling only, can become messy without proper structure	Web design, UI styling, responsive layouts

C++	High-level, compiled	High performance, control over memory, object-oriented	Complex syntax, steep learning curve, risk of memory errors	Game development, system software, performance-critical applications
JavaScript	High-level, interpreted	Runs in browsers, versatile, large ecosystem	Can be inconsistent across browsers, not as fast as compiled languages	Web development, interactive UI, server-side (Node.js)

I decided to choose Python since I already have a good level of experience with using it, which has given me a good base for coding and problem-solving. From my projects, I am used to coding clean, efficient code and leveraging Python's library foundation to accomplish everything from data analysis to automations. My experience with Python also involves coding within Visual Studio Code, where I have learned about debugging, versioning and environment customisation through extensions. This is second nature to me when it comes to Python because it is the capability of learning from what you already know while still expanding in terms of programming capability.

Comparison Of IDE's

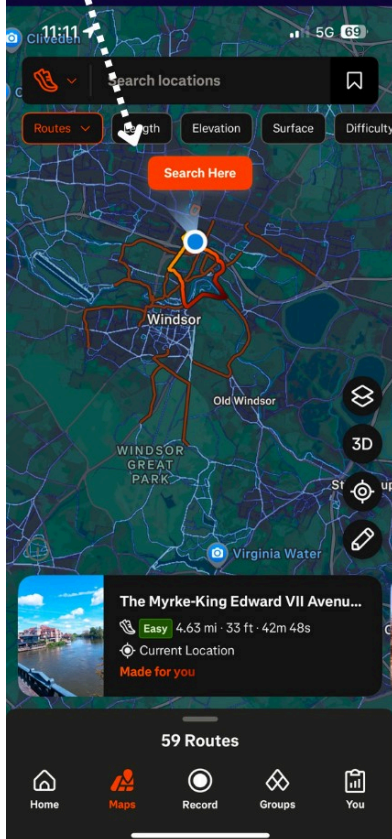
<u>IDE</u>	<u>Language s Supporte d</u>	<u>Key Features</u>	<u>Pros</u>	<u>Cons</u>
<u>Visual Studio Code (VSC)</u>	Python, JavaScript, Java, C++, C#, HTML/CSS and more	Lightweight, extensions marketplace, integrated terminal, debugging tools, Git integration	Free, cross-platform, highly customizable, strong community support	Can become resource-heavy with too many extensions
<u>PyCharm</u>	Python, JavaScript, HTML/CSS	Smart code navigation, scientific tools, an integrated debugger, and version control	Excellent for Python, professional tools, productivity boosters	Resource-inten sive, full version is paid.

<u>IntelliJ IDEA</u>	Java, Kotlin, Groovy, SQL and more	Code completion, refactoring, version control, and debugging	Great for JVM languages, powerful refactoring tools	Heavy on resources, the full version is paid
<u>Eclipse</u>	Java, C, C++, Python (via plugins)	Plugin system, debugging, and project management	Open-source, powerful for Java, lots of plugins	UI feels outdated, and can be slow.

In comparing all the different IDEs, such as PyCharm, IntelliJ IDEA and Eclipse, it was clear that each was matched up against certain programming needs. For example, PyCharm was excellent with Python but might be resource-hungry, IntelliJ IDEA was extremely powerful for Java development but with a steep learning curve, and Eclipse had wide plugin coverage but is a bit outdated in UI. Or, Visual Studio Code (VSC) is a light yet highly adaptable option that supports an extensive range of languages via a massive extensions market. It's highly effective for both learning and production environments with its integrated terminal, debugger and seamless Git integration. Because of its flexibility, excellent community support and also because of the first-hand experience that I have with VSC to use Python, I employed it as my primary IDE.

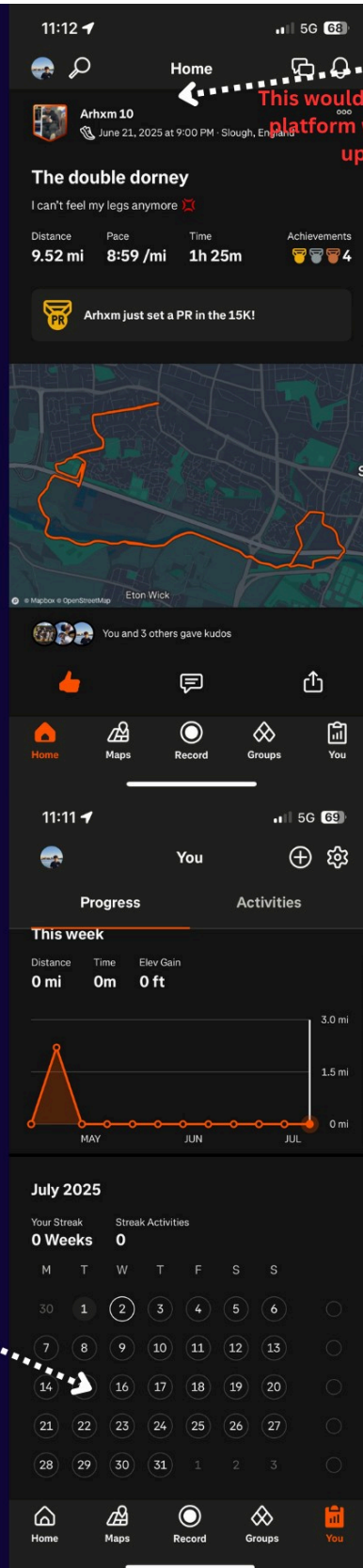
Map Implementation

I will use something similar to this however more catered towards finding others in your chosen proximity



Fitness Date Tracker

Again i will implement something similar to this in my platform but you will be able to make annotations to the days where things like checkpoints or to do lists related to martial arts can be added.

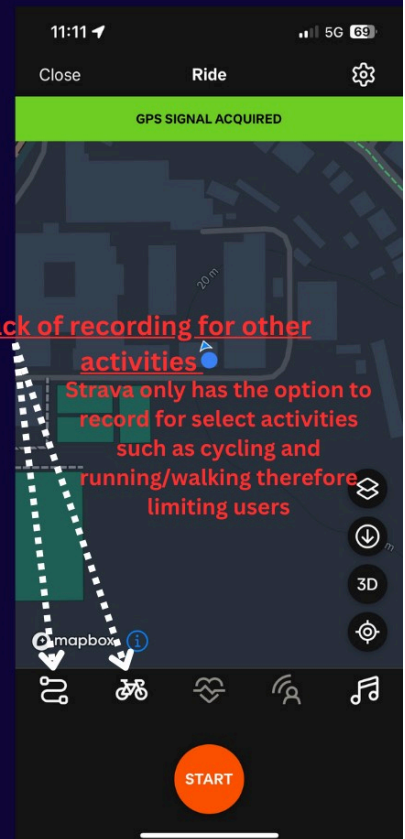


Contains a social aspect

This would be a good feature to implement into my platform where others can friend each other and upload/share their achievements.

Lack of recording for other activities

Strava only has the option to record for select activities such as cycling and running/walking therefore limiting users



Search sparring partners by:

State
Please select location

Age Range
Please select

Weight class
Please select

Height
Please select

Stance
☐ Right-Handed ☐ Left-Handed

Number of fights
Please select

Gender
☐ Male ☐ Female

Search Function
Fightura has a search function which will be very useful for me to add to the list of functions for my platform, however there will be a proximity feature where a map will be displayed and a certain radius/diameter can be selected.

User Register
Fightura also has a register function which has multiple sections to fill out so that some information can be displayed on your profile and some private

Name
Name

Gender
☒ Male ☐ Female

Height
Please select

Weight class
Please select

Stance
☒ Right-Handed ☐ Left-Handed

Number of fights
Amateur: 0 Professional: 0

Date of birth
MM DD YYYY

City
City

State / Province
Please select

Gym name
Gym name (optional)

Your Rate
\$ US Dollar (optional) .00

Tell us more about yourself
Tell us more about yourself (optional)

Email
Email

Password
Password

Confirm password
Confirm password

Register

Filters

Depending on the location the relevant coaches/fighters or gyms will be displayed. There is also a weight and height filter along with experience, stance, discipline and levels this gives the user a more fine tuned option for searching.

Different Options For Registration

You can register as a trainer which can unlock other features such as posting specific posts however this option doesn't consist of a verification method thus letting anyone to register as a trainer which can lead to further issues.

All Genders

Region
All Regions

Disciplines
Select a Discipline

Stance
Any Stance

Experience
All Experience Levels

Level
All Levels

Height (cm)
Slider

Weight (kg)
Slider

Discipline	Age	Weight	Experience
Thai Boxing	28	64kg	Intermediate
Boxing	35	69kg	

Discipline	Age	Weight
Boxing	48	75kg
MMA	26	87kg

Discipline	Age	Weight	Experience
Boxing	48	75kg	Beginner

Discipline	Age	Weight
Boxing	48	75kg
MMA	26	87kg

All Boxing / MMA Gyms

Search gyms by state:
Please select state

All
Alabama
Alaska
Alberia
Arizona
Arkansas
British Columbia
California
Colorado
Connecticut
District of Columbia
Delaware
Florida
Georgia
Hawaii
Idaho



12th Avenue Boxing
Address: 5234 S 12th Ave
Tucson, AZ 85706
Phone: (520) 306-1900
[View Details](#)



5th St Gym
Address: 1434 Alton Rd
Miami Beach, FL 33139
Phone: (305) 763-8110
[View Details](#)



Atlantic City Police Athletic League
Address: 250 N New York Ave
Atlantic City, NJ 08401
Phone: (609) 347-5478
[View Details](#)



American Top Team

Lack of locations

There are only few locations which can be searched for which are limited to the US, this can be improved by adding more locations which can be global.

Success Criteria

<u>Objective</u>	<u>Explanation</u>	<u>Success Criteria</u>	<u>Sub-Success Criteria</u>	<u>Priority</u>
1) Must allow a new user to sign up	The program must allow a new user to create an account unless the email or username has already been used.	1a) Program must provide access to the sign-up page	1ai) A visible button must navigate the user to the sign-up page from the login page	HIGH
			1aii) The sign-up page must load with all input elements visible and enabled	HIGH
		1b) User must be able to enter registration details	1bi) Designated textbox for username	HIGH
			1bii) Designated textbox for email address	HIGH
			1biii) Designated textbox for password	HIGH
			1biv) Drop-down selection for martial art	MED
			1bv) Drop-down selection for skill level	MED

			1bvi) Button labelled "Sign Up"	HIGH
		1c) Program must prevent duplicate accounts	1ci) If the email already exists in the database, sign-up must be denied	HIGH
			1cii) Message displayed: "This email is already linked to an existing account. Please log in."	HIGH
		1d) Program must confirm account creation	1di) User details must be stored in the database upon successful sign-up	HIGH
			1dii) Message displayed: "Account successfully created. Please log in."	HIGH
			1diii) If database connection fails, display an error message	MED
2) Program must allow users to log in	All registered users must be able to log in using valid credentials.	2a) Access to login page	2ai) Program initially displays the login page	HIGH
			2aii) User can navigate to login	HIGH

			page from sign-up page	
		2b) Credential validation	2bi) Single textbox accepts username or email	HIGH
			2bii) Program distinguishes between email and username input	HIGH
		2c) Incorrect login handling	2ci) Access denied if password does not match database	HIGH
			2cii) Message displayed: "The username, email or password you entered is incorrect."	HIGH
			2ciii) User is allowed to retry login	HIGH
		2d) Successful login	2di) User is redirected to dashboard upon successful login	HIGH
			2dii) Login form is hidden after authentication	HIGH

3) Display user dashboard	After login, the program must display a personalised dashboard.	3a) Dashboard information	3ai) Username displayed on dashboard	HIGH
			3aii) Martial art and skill level displayed	HIGH
		3b) Dashboard navigation	3bi) Button to search for training partners	HIGH
			3bii) Button to search for gyms	HIGH
			3biii) Button to search for coaches and medical professionals	HIGH
			3biv) Button to access messages	HIGH
4) Allow user to search for training partners	Users must be able to find suitable training partners based on relevant criteria.	4a) Search filters	4ai) Filter by martial art	HIGH
			4aii) Filter by skill level	HIGH
			4aiii) Filter by location	HIGH

		4b) Search results	4bi) Results match selected filters only	HIGH
			4bii) Each result displays username, skill level, martial art and location	HIGH
			4biii) Message displayed if no results are found	MED
5) Provide services directory	Users must be able to locate gyms, coaches and medical professionals.	5a) Service filtering	5ai) Filter by service type (Gym, Coach, Medical)	HIGH
			5aii) Filter by martial art specialism	MED
			5aiii) Filter by location	HIGH
		5b) Service display	5bi) Each listing displays name, type and location	HIGH
			5bii) Contact or message option available	HIGH
6) Allow secure messaging	Users must be able to communicate to arrange training or services.	6a) Message creation	6ai) User can send messages to another user	HIGH

			6a ii) Message stored in the database	HIGH
		6b) Message display	6bi) Messages display sender, content and timestamp	HIGH
			6bii) Messages displayed in chronological order	MED
7) Allow profile management	Users must be able to edit their personal training information.	7a) Profile editing	7ai) User can edit martial art and skill level	HIGH
			7a ii) Changes saved to database	HIGH
			7a iii) Updated information displayed immediately	MED
8) Program must be usable and visually clear	The system must be easy to navigate and aesthetically consistent.	8a) Interface consistency	8ai) Consistent layout across all pages	HIGH
			8a ii) Text is readable and appropriately sized	HIGH
			8a iii) Navigation is intuitive with clear labels	HIGH

			8aiv) Platform usable on desktop and mobile devices	MED
--	--	--	---	-----

Referential Integrity and Database Design (BrawlBase)

Referential Integrity and the Need for Structured Relationships

One of the important problems which can come up during the coding of the BrawlBase program is referential integrity. Of all the aspects of relational database management, one of the key ideas is ensuring referential integrity. This is defined as ensuring that all relationships between the tables in a database are valid. This is equivalent to stating that all values represented in the foreign keys must also exist as values represented in the parent table where the foreign key is represented, or be NULL.

If referential integrity is not enforced, inconsistencies may occur. For example, a foreign key may reference a value that does not exist in the parent table. This situation can arise through incorrect data entry, poor validation, or the deletion of a record that is still being referenced by another table. In the context of BrawlBase, this could lead to scenarios where a training session appears to be linked to a user who no longer exists, or a service interaction references a gym or coach that has been removed from the system. Inconsistencies at these points would strongly lower the reliability of the system. BrawlBase is designed to connect martial artists with training partners, gyms, coaches, and medical professionals, meaning that accurate relationships between users and services are critical. Without referential integrity, search results can turn misleading, user interactions can reference invalid data, and this reduces the trust in the platform as a whole.

In order to avoid these problems, referential integrity will be applied across the database design. This will be achieved by structuring the tables as needed, along with the correct position of primary keys, foreign keys, and their constraints to ensure a valid relationship status. Many-to-many relationships will not be encouraged and, rather, will be handled by the junction table. The integrity of the referral that is being used within the database level will also eliminate the need or reliance offered by just using application level validation alone. Though application-level programming of checking is required, constraints placed from within the databases are a measure of checking and ensuring that no incorrect data enters a database.

Core Tables and Resolution of Many-to-Many Relationships

The BrawlBase database design starts with the development of key tables that will hold key system information. The first key table in the database design of BrawlBase will be the User table. The User table will hold basic information that is relevant to users of the BrawlBase system. The information will include key attributes such as a user ID, username, email address, martial art discipline, and level of skill. The User table will utilise a single key to uniquely identify users in the database.

Another main table, Training_Session, exists to collect data regarding individual training activities. This table comprises additional data fields like type, date, location, and discipline. Similar to the User table, this table also contains a single primary key, with no relationships with any other data table. All these main tables, being independent, reduce data complexity with the involvement of multiple relationships. Accordingly, a single user may be involved in many training sessions. Also, a single training session may be attended by many users. This is a many-to-many relationship, which cannot be stored in a relational database without redundancy. This is where a new table named Completed_Sessions is included. In the design of the database using BrawlBase, there is the development of basic tables that contain basic information about the system. The first table in the BrawlBase is the User table. In the User table, there are basic pieces of information about each user in the system. These pieces of information include a specific user ID, username, email, martial art discipline, and skill level. The User table contains a single key that uniquely identifies a user in the system.

Another key table used in conjunction with storing information related to training sessions involving individuals is the training session table known as Training_Session. It has attributes like the nature of the training session, the date, the location, and the discipline associated with the training session. It has only a single key and does not relate to any other tables as a key in the User table. Practically speaking, there can be many training sessions for a single user, and a single training session can be attended by many users. However, such a scenario is not possible within a relational database as it is considered redundant or violates the principles of normalisation. To solve this dilemma, a third table named "Completed_Sessions" is used.

The table Completed_Sessions is an intersection table that also has a candidate key associated with it. There are two foreign keys for the table. One of them is related to the User table, whereas the other is related to the Training_Session table. The above scheme ensures that all completed sessions have a legitimate user as well as a legitimate training session. As constraints of referential integrity have been defined on the database tables, it is not possible for an invalid or non-existent user and session to be added to the table. An Entity Relationship Diagram has been designed to easily create the visual representation of the database. This diagram is designed to obtain an overview of all the tables included in the database. This has been done using free online tools such as draw.io for visualising the database scheme easily. The relationship between the entities in the ERD follows the Barker notation, a type of crow's foot notation. This notation was used because it is easy to read and can clearly distinguish between a one-to-one, one-to-many, and many-to-many relationship. From the ERD created, one can see that the relationship between a user and

the completed session allows a user to have many completed sessions, and one completed session may have many users using the information in the Completed_Sessions relationship. Therefore, this ensures that the resolution of the many-to-many relationship was done correctly.

Additional Tables, One-to-One Relationships, and Design Justification

Apart from training-related data, BrawlBase also needs to accommodate data regarding gyms, coaches, and medical professionals. For further substantiation of such tables, two new tables, namely Service and Service_Interaction, are defined. These tables consist of information regarding gyms, coaches, and medical professionals, followed by services and user interactions with such services, such as viewing profiles and making contact. These tables will be related to the User table through foreign key columns. Hence, such interactions will be associated with an existing user. There will be no possibility of service interactions relating to users that do not exist anymore through referential integrity in the database. A further table, User_Profile, will also be designed, which stores information like data on mode of training, availability, and location, all of which are optional. This table will have a one-to-one relationship with the User table. Each user can have at most one corresponding record in the User_Profile table, and each profile entry belongs to exactly one user.

Although it would be possible to store all user-related information in a single table, this approach was deliberately avoided. Many of the attributes stored in User_Profile are optional, and placing them in the User table would result in a large number of NULL values. Separating this data improves normalisation, reduces wasted storage space, and improves query efficiency. If a user chooses not to provide additional profile information, the User_Profile table is never referenced, further optimising storage. In the User_Profile table, the primary key field is also a foreign key, and its primary key refers to that of the User table. Therefore, this is not a normalisation flaw, as the relationship under consideration between a user and a user profile in this case can exist individually. In a one-to-many relationship, a primary key must exist.

This stage, therefore, creates a sense of purpose as to why the data models are being built in the analysis stage. This sense of purpose will, in turn, ensure that the different stages of the project, i.e., design and implementation, are done with a level of structure, thus preventing errors. This, in turn, will ensure that the BrawlBase database project will be effective as desired.

Questionnaire – User Research for BrawlBase

Purpose of the Questionnaire

The purpose of this questionnaire was to gather primary research from a martial artist in order to better understand the difficulties faced when finding training partners, gyms, coaches, and medical professionals. The responses were used to identify common problems and directly inform the functional and non-functional requirements of the proposed system.

Question 1

How easy do you find it to locate suitable martial arts gyms when travelling or training in new areas?

- Very difficult
- **Difficult**
- Neutral
- Easy
- Very easy

Response and Analysis:

The respondent mentioned it is hard to find gyms, especially when traveling. Currently, platforms such as Fightura and Spartner are available, but they do not allow users to effectively search for gyms based on a specific discipline of martial arts, which is a necessity for users and thus a requirement for our application.

Question 2

How difficult is it to find training partners who match your skill level?

- **Very difficult**
- Difficult
- Neutral
- Easy
- Very easy

Response and Analysis:

The respondent reported that it was very difficult to find opponents who were at the same level or one level above them in terms of skill level. This was mainly because there were few mixed martial arts gyms within the locality where the respondent was from. Fighting with people who were one level below made training more difficult, which validated the need for such a system based on skill level and kind.

Question 3

How easy is it to find medical professionals who specialise in martial arts-related injuries (e.g. physiotherapists, sports therapists)?

- Very difficult
- Difficult
- Neutral
- Easy
- Very easy

Response and Analysis:

The respondent indicated that this was a very difficult process. This is because it is understood that injuries occur frequently in martial arts. A lack of access to specialists in combat sports was highlighted as time-consuming. The injuries sustained have been highlighted as still causing problems currently since access to rehabilitative care was limited. This is to illustrate the need for the presence of medical professionals.

Question 4

How difficult is it to find coaches who are suited to your training needs and skill level?

- Very difficult
- Difficult
- Neutral
- Easy
- Very easy

Response and Analysis:

The respondent indicated that it is difficult to find suitable coaches, as most gyms have a small number of coaches who may not match specific training needs. This supports the inclusion of a searchable coach directory within the system.

Question 5

Are you currently aware of any existing websites or applications that fully meet your training needs?

- Yes
- No

Response and Analysis:

The respondent answered No, explaining that although such platforms exist, for example, Fightura or Spartner, they do not have essential features and are mostly used by beginners or less experienced practitioners of martial arts in general. This affirms the assertion that the solutions are not sufficient for sophisticated users.

Question 6

Would you find value in receiving personalised feedback based on your training activity and consistency?

- Yes
- No

Response and Analysis:

The respondent answered Yes, which indicates an interest in obtaining feedback in regards to training. Thus, it seems that certain attributes, such as activity tracking or even feedback, might have a role in encouraging engagement.

Question 7

Do you record your sparring or training sessions, and would you benefit from a platform where this content could be reviewed by others?

- Yes
- No

Response and Analysis:

The answer from the respondent was affirmative, stating, “Yes, because I film myself during training sessions, and I will be able to benefit from uploading this footage in order to obtain feedback from coaches or from people who are experienced in this field.” This is relevant to further expansion and community feedback.

Question 8

Do you currently use any platforms to track your fitness activity?

- Yes
- No

Response and Analysis:

The respondent answered Yes, mentioning the use of MyFitnessPal and Strava. However, these were described as being poorly suited to martial arts, since they focus primarily on calorie tracking, running, or cycling. Again, this indicates a need for a solution particular to martial arts.

Summary of Findings

The data gathered by the questionnaire shows some prominent issues, and one can find a trend that individuals all have trouble finding training partners, coaches, and medical experts skilled at a level commensurate with their ability, as well as overall dissatisfaction with existing relevant platforms. Such issues are directly relevant to the creation of BrawlBase and inform requirements and design decisions in the creation of this system.

BrawlBase - Design

Design Overview

The purpose of this design phase is to satisfy the requirement of translating the relevant requirements gathered in the Analysis phase into a detailed and systematic design for the BrawlBase system. System design entails a clear and logical approach to how the system will be developed before programming begins, considering all the relevant and non-relevant requirements, whether functional or non-functional. The proposed system, i.e., BrawlBase, is maintained as a web-based system that facilitates martial artists to get connected with training partners, gyms, coaches, and even medical staff based on martial art type, training level, and geographical location. To enable all these requirements, the system being proposed is divided into different parts, each covering a different requirement of a martial artist.

The system consists of the following basic elements:

- A user authentication system to securely register and login
- A relational database for user profiles, training sessions, services, and messages
- A search-and-filter system to help users locate training partners or service providers.
- A messaging system enabling communication between users
- A user interface that is intuitive, accessible, and perfect for all devices

Each of these components has been made traceable. Every design decision can be linked back directly to information gathered in the Analysis phase, such as the results of the questionnaires, problem areas, and success criteria. This ensures that the system design is user-centred and purpose-driven rather than feature-driven.

For example, a module-based approach has been followed in the overall system design. The overall idea behind this is that a module tends to have a separate role to perform within the overall system design. These modules exchange information using data flow techniques with the other modules in the system design. For example, the authentication module works independently of the search module, but both modules are dependent on the same data for users, which resides in the database. This approach ensures that the system design is need-driven and user-centric instead of being feature-driven. A modular approach has been followed for overall system design, and each module has been given a specific responsibility while interacting with other modules of the system through data flow. It helps in maintaining and simplifying the system by making components of each module independently testable during the development phase itself. For example, the authentication module and the search module operate independently of each other, though they both use the same data present in the database for users.

The design also takes into account non-functional requirements such as usability, performance, and data integrity. The user interface is designed to minimise the number of interactions required to complete common tasks, such as finding a training partner or

contacting a coach. Database design decisions prioritise referential integrity and normalisation to ensure data accuracy and consistency. The subsequent sections in the design phase will outline in detail all the components of the system. This will include database design and entity relationship diagrams, algorithm design using structured pseudocode, and the user interface designs, which will be further backed up with wireframes and more. This will essentially act as a detailed blueprint for the development phase for Brawl Base.

Database Design Overview

The BrawlBase system relies heavily on the storage, retrieval, and management of structured data. In order to support key features such as user authentication, training partner matching, service discovery, and messaging, a relational database model has been chosen. This approach is well-suited to the nature of the data involved, as it allows relationships between users, training activities, and services to be clearly defined and enforced. A relational database provides several advantages that are essential for BrawlBase. Firstly, it allows data to be stored in a structured and organised manner using tables, rows, and columns. Secondly, it supports the use of primary keys and foreign keys, which are necessary to maintain referential integrity and ensure that all relationships between data entities remain valid. Finally, relational databases support complex queries, which are required for filtering users and services by criteria such as skill level, discipline, and location.

The database structure has fully taken into account the concept of normalisation, which helps to reduce data redundancy as well as increase consistency. Normalisation ensures that the data that has been entered into the database is not redundant, as data entered into this database is stored in a manner where there are no duplication issues, with all the data entries being where the data is actually required. All data will be unique, which is a requirement. One of the key considerations during the design of the BrawlBase database is referential integrity. All relationships between tables should be enforced through foreign key constraints; records should not be able to reference data that does not exist. This will be especially important for a platform such as BrawlBase, since invalid links between users, training sessions, or services could lead to meaningless search results or other broken functionality. The system remains robust even in the case of application-level errors by enforcing referential integrity at the database level.

The database design avoids unresolved many-to-many relationships by introducing junction tables where appropriate. For example, a user may participate in many training sessions, and a single training session may involve multiple users. Instead of holding duplicate data, a junction table is utilised to connect these entities in a controlled fashion. In this way, the data system becomes more flexible, and the data system becomes easier to extend without requiring radical redesigns. The overall structure of the database has been planned before implementation in order to ensure that the database meets all the functional requirements specified during the Analysis phase of the model. Each data table in the database has been designed in such a manner that it can match directly with any of the success criteria specified during the Analysis phase of the model. For example, in the database designed during this case study, the need to search training partners by skill levels has been addressed while maintaining the user-related tables in the database.

An Entity Relationship Diagram (ERD) has been used to visually represent the database structure. The ERD shows each table, its attributes, and the relationships between tables. This diagram, at the same time, serves as a design blueprint, and it is easier to validate that the reality, in fact, has been correctly reflected in the database design. The ERD will also help point out potential issues, like unneeded relationships or missing relationships, well before they actually take place. Each of the above-mentioned sections provides an in-depth account of all the database tables with respect to their use, nature, primary key, and any possible foreign key attributes. This ensures that the design of the database is logically correct and well understood concerning the overall objectives of the system, which is BrawlBase.

Junction Tables and Relationship Resolution

Purpose of Junction Tables in BrawlBase

In BrawlBase, many of the real-world relationships are by their nature many-to-many. Thus, a user may train in several gyms, and a gym may have many users. Likewise, a user can interact with multiple coaches or medical professionals, but each professional may interact with many users. Within a relational database, it is not possible to implement many-to-many directly without data redundancy, inconsistencies, or violation of normalisation principles.

In order to resolve such relationships without compromising data integrity, junction tables are used. A junction table consists of foreign keys that refer to the primary keys of two parent tables. Each record of the junction table shows a valid association of those two entities. This structure reduces duplication; the database enforces referential integrity through foreign key constraints. Using junction tables also increases scalability. In case additional data of the relationships must be recorded sometime later, such as the date a user joined the gym or the type of interaction with the service, attributes are added to the junction table without changing the parent tables.

Many-to-Many Relationships Identified

The following many-to-many relationships were identified during the design of BrawlBase:

- Users and gyms: a user may train at multiple gyms, and a gym may have many users
- Users and coaches or medical professionals: a user may interact with multiple professionals, and each professional may interact with many users
- Users and users through messaging: users can send messages to many other users and receive messages from many senders

These relationships require careful handling to ensure the database remains normalised and that all references remain valid.

UserGym Junction Table (Users and Gyms)

Purpose

The **UserGym** table records the relationship between users and gyms. This allows the system to store which gyms a user trains at and supports features such as displaying gym memberships and recommending gyms based on user activity.

Attributes

Field Name	Data Type	Key	Description
UserGymID	Integer	Primary	Unique identifier for each user-gym relationship
UserID	Integer	Foreign	References User(UserID)
GymID	Integer	Foreign	References Gym(GymID)
DateJoined	DateTime		Stores when the user is associated with the gym

Relationship Justification

This design resolves the many-to-many relationship between users and gyms by storing each association as a separate record. Referential integrity ensures that a UserGym record cannot exist unless both the referenced user and gym exist in the database.

UserServiceInteraction Junction Table (Users and Services)

Purpose

BrawlBase provides access to gyms, coaches, and medical professionals through a services directory. Users may interact with multiple services, and each service may be interacted with by many users. The **UserServiceInteraction** table records these interactions in a structured way.

Attributes

Field Name	Data Type	Key	Description
InteractionID	Integer	Primary	Unique identifier for each interaction
UserID	Integer	Foreign	References User(UserID)
ServiceID	Integer	Foreign	References Service(ServiceID)
InteractionType	Varchar		Type of interaction (e.g. saved, contacted)
Timestamp	DateTime		Date and time of the interaction

Relationship Justification

This table prevents duplication of service data while allowing multiple interactions to be recorded between users and services. It also allows interaction history to be stored without modifying the service or user tables.

Messaging as an Associative Entity (Users and Users)

Although messaging involves a many-to-many relationship, a traditional junction table is not required because each message is a distinct entity. Instead, the **Message** table acts as an associative entity linking two users: the sender and the receiver.

Attributes

Field Name	Data Type	Key	Description
MessageID	Integer	Primary	Unique identifier for each message
SenderUserID	Integer	Foreign	References User(UserID)
ReceiverUserID	Integer	Foreign	References User(UserID)
Content	Text		Message content
Timestamp	DateTime		The time the message was sent
Status	Varchar		Message state (e.g. sent, read)

Referential Integrity

Referential integrity ensures that a message cannot exist unless both the sender and receiver are valid users. Appropriate delete rules can be applied to prevent orphaned records if a user account is removed.

Enforcement of Referential Integrity

To ensure consistency of the data across all relationships contained in BrawlBase:

- The foreign key constraints will apply to all fields involved in the relationships
- Relationship records cannot exist without parent records in place.
- The deletion rules will be implemented with consideration, for instance, for gym deletion rules, consideration of gyms still being referenced will be made
- This ensures that results, user interaction, and messages remain accurate and reliable.

Link to Requirements and Success Criteria

The use of junction tables has a direct impact on several of the functional requirements and success factors identified in the analysis phase. This includes providing users with the ability to associate with many gyms as well as services, providing a reliable messaging service, and providing consistent and valid data relationships.

This ensures that BrawlBase will be scalable, maintainable, and implementable while keeping the integrity of the data intact.

Data Structures and Database Schema

Database Schema Table:

Entity	Attribute	Type	Key	Description
User	UserID	Integer	Primary	Unique identifier for each user
User	Username	Varchar		Chosen username
User	PasswordHash	Varchar		Encrypted password storage
User	MartialArt	Varchar		Type of martial art practised
Coach	CoachID	Integer	Primary	Unique identifier for each coach
Coach	Specialism	Varchar		Area of martial arts expertise
Gym	GymID	Integer	Primary	Unique identifier for each gym
Gym	Facilities	Text		Available facilities in the gym
Professional	ProfID	Integer	Primary	Unique identifier for

				each professional
Professional	Profession	Varchar		Roles such as a physiotherapist or a nutritionist

Testing Strategy

Test Plan Table:

Test Case	Input Data	Expected Result	Type
Log in with valid details	Username=Ahmed, Password=12345	Login successful	Normal
Log in with the wrong password	Username=Ahmed, Password=Wrong	Error message shown	Erroneous
Search by skill level	Skill=Intermediate	List of intermediate users	Normal
Search with an invalid filter	Skill=UltraMaster	No results returned	Erroneous
Boundary test password length	Password=5 chars	Error: too short	Boundary

Algorithms

The functionality of BrawlBase relies on a set of core algorithms that ensure the platform runs smoothly, delivers accurate results and provides a positive user experience. Each algorithm has been designed to balance efficiency with simplicity so that it can be implemented in Python and extended in future updates. Below are the main algorithms with their design, pseudocode and explanation.

Login Algorithm

Pseudocode:

```

FUNCTION signin(user, passkey):
    userRecord = database.find(user)
    IF userRecord IS NOT NULL:
        hashedInput = hash(passkey)
        IF hashedInput == userRecord.passkeyHash:
            RETURN 'Proceed'
        ELSE:
            RETURN 'Wrong password'
    ELSE:

```



```

    RETURN 'User not in database'
END FUNCTION

```

This algorithm checks whether a username exists in the database and validates the entered password against the stored hash. It returns either success or an appropriate error message.

Registration Algorithm

Pseudocode:

```

FUNCTION register(username, password, martialArt, skillLevel, location):
    IF database.find(username) IS NOT NULL:
        RETURN 'Username already exists'
    IF length(password) < 6:
        RETURN 'Password too short'
    hashedPassword = hash(password)
    newUser = User(username, hashedPassword, martialArt, skillLevel, location)
    database.insert(newUser)
    RETURN 'Registration successful'
END FUNCTION

```

This algorithm ensures new users create unique accounts and that their data is valid. It prevents duplicate accounts, validates password strength, hashes the password before saving and ensures key user details are stored.

Search Algorithm

Pseudocode:

```

FUNCTION search(criteria):
    results = []
    FOR database entry:
        IF entry matches criteria:
            results.append(entry)
    RETURN results
END FUNCTION

```

This algorithm filters users, coaches, or gyms based on selected criteria such as martial art, location and skill level. It can be extended to include sorting by distance, reviews, or availability.

Recommendation Algorithm

Pseudocode:

```

FUNCTION recommendPartners(userProfile):
    recommendations = []
    FOR candidate IN database:
        IF candidate.martialArt == userProfile.martialArt:
            IF abs(candidate.skillLevel - userProfile.skillLevel) <= 1:
                IF candidate.location == userProfile.location:

```

```

        recommendations.append(candidate)
    RETURN recommendations
END FUNCTION

```

This algorithm matches martial artists with others of similar skill and location, making connections more relevant. It prioritises discipline, skill and geographical proximity.

Messaging Algorithm

Pseudocode:

```

FUNCTION sendMessage(sender, receiver, content):
    IF database.find(sender) IS NULL OR database.find(receiver) IS NULL:
        RETURN 'User not found'
    message = { 'from': sender, 'to': receiver, 'content': content, 'timestamp': now() }
    database.save(message)
    RETURN 'Message sent'
END FUNCTION

```

User Interface Design

The user interface (UI) of BrawlBase is created with clarity, consistency and accessibility in mind. As most users will be accessing the website on mobile devices, a mobile-first approach was used for the website. Each screen has been made easy and intuitive, with the least number of clicks needed to access basic functionalities.

General Design Principles

- Consistency: Same colour scheme, fonts and button types on all screens.
- Minimalism: Few features shown per page to avoid clutter.
- Accessibility: Large buttons, screen reader text labels, high contrast colour.
- Responsiveness: Automatically resizing pages for mobile, tablet and desktop

Key Screens

Login Page

Fields: Username, Password.

Buttons: Login and Register.

Features: Error messages for incorrect login and password reset link.

Wireframe:

```

-----
|   BrawlBase Logo   |
|-----|
| Username: [        ] |
| Password: [        ] |
|                   |
| [ Login ] [ Register ]|
| Forgot Password?    |
|                   |
|-----|

```

Dashboard

After login, users see a personalised dashboard.

Options: Search for fighters/gyms, Messages, Profile, Services.

Quick access buttons for most-used features.

Wireframe:

```

-----
|   Welcome, Ahmed   |
|-----|
| [ Search Partners ] |
| [ My Messages ]    |
| [ My Profile ]     |
| [ Services Directory ]|
|                   |
|-----|

```

Search Results Page

Input: Filters for location, martial art, and skill level.

Output: List of users/gyms with basic info.

Feature: Connect button to send requests.

Wireframe:

```

-----
| Filters: Martial Art, |
| Skill, Location      |
|-----|
| AhmedK | Boxing | Int |
| [Connect]| Location: SI | |
|---|---|---|
| SaadMMA | MMA | Adv |
| [Connect]| Location: Ldn|
|-----|

```

Profile Page

Shows user info: martial arts, skill level, connections.

Allows editing of details and uploading of training footage.

Option to view reviews left by others.

Messaging Page

Simple chat layout.

Conversation history with timestamps.

Option to report/block users for safety.

Accessibility Features

- Font sizes will be adjustable.
- Colour scheme chosen keeping accessibility as an option (red/green clashes avoided).
- Alternating text on images.
- Error messages explained clearly.

Justification

This design was chosen because it is minimalist yet functional. Users can quickly find training partners, coaches, or services without navigating through unnecessary menus. The wireframes reflect feedback gathered from the questionnaire, ensuring that usability is prioritised.

BrawlBase - Security and Validation

Security and Validation

Security and validation are necessary while designing BrawlBase. Since the site will be handling sensitive data such as usernames, password hashes and personal data of fighters, best practices need to be adopted so that the information of the user remains secure and unauthorised use is not made of it.

Password Hashing Methods

Plain text storage of passwords would be very insecure. If the database is ever breached, attackers could simply access accounts. To prevent this, BrawlBase will use password hashing:

- Hashing: A one-directional method that applies an algorithm to transform data of any size into a fixed-length string of characters that uniquely represents it.

Input Validation Rules

All the inputs from the user need to be validated to prevent malicious or corrupt data from being entered into the system. Validation will be done on both the client side (to notify the user right away) and the server side (for security) as well.

Examples:

- Username Validation: Must be unique, length 3–20 characters, no spaces.
- Password Validation: Minimum 6-character length, letters and numbers must be present.
- Email Validation: It must have '@' and a correct domain name.
- Location Validation: Must follow familiar patterns (i.e., city names, postcodes).
- File Upload Validation: Training video uploading should be limited to a specific type and size (e.g., avi, mp4 only).

Prevention of Common Attacks

BrawlBase will be vulnerable to standard web-based attacks. Countermeasures will be incorporated into the design:

- SQL Injection: Prevented with parameterised queries (treating entered information as data instead of code).
- Brute Force Attacks: Login will be limited to a limited number (e.g., 5 attempts before a temporary lockout).
- Session Hijacking: Secure session tokens will be generated, and logging out when idle will be enabled.

Error Handling

Error messages must be both user-friendly and human-readable, but not reveal sensitive system details. For example:

'Invalid password' instead of 'Password does not match for username Ahmed.'

'User not found' instead of 'No record exists in the User table.'

This will make it difficult for the attacker to gain valuable information about the database or system.

BrawlBase - Testing Strategy

Testing Strategy

Testing is essential in ensuring that BrawlBase functions as expected and meets the requirements outlined during the analysis phase. A clear strategy has been developed that covers different types of testing and the use of structured test cases.

Unit testing

Unit testing focuses on individual modules. Each function, such as login, registration, or search, will be tested separately to confirm that it performs its role correctly. This makes it easier to locate errors at an early stage.

Integration testing

Integration testing is used once modules are combined. It checks whether the login system correctly links to the profile module or whether the search module communicates with the database as intended. This ensures a smooth flow of data between different parts of the system.

User acceptance testing

User acceptance testing involves the end user to ensure that the system meets the needs originally identified. This will be carried out by providing the completed application to martial artists and coaches who will test the platform against real-world expectations.

Test plan

A structured test plan is used to check functionality. This covers normal data, boundary data and erroneous data.

Test Case	Input Data	Expected Result	Type
Log in with valid details	Username=Ahmed, Password=Secure123	Login successful	Normal
Log in with an incorrect password	Username=Ahmed, Password=WrongPass	Error message shown	Erroneous
Registration with a duplicate user	Username=Ahmed, Password=NewPass	Error: Username already exists	Erroneous
Registration password boundary	Password=5 characters	Error: Password too short	Boundary
Search by martial art	MartialArt=Boxing	List of users who practice boxing	Normal

<i>Search with an invalid filter</i>	MartialArt=UltraMaster	No results returned	Erroneous
<i>Upload training video</i>	File=video.mp4 size=50MB	File uploaded successfully	Normal
<i>Upload the wrong file type</i>	File=document.pdf	Error: Invalid file type	Erroneous

Sample test cases explained.

A normal case would be logging in with the correct details, which confirms that the login module works under expected conditions.

A boundary test would be attempting to sign up with a password that is shorter than the minimum stated and seeing whether the validation rules are valid.

User_Profile Table Design

Purpose of the User_Profile Table

The purpose of the User_Profile table is to store optional data about the user that is not considered during user registration or authorisation. By creating separate tables for this kind of data, the normalisation of the database is improved. Also, the problem of NULL in the User table is removed.

This ensures that the underlying User table in the database does not become overly complicated with added information while still allowing users to add the information that they believe will make the matching more effective and user-friendly. There might be additional information, such as training preferences that users can add; however, users should not have to add additional information in order to make full use of the functionality.

User_Profile Table Attributes

The table below shows the fields that will be stored in the User_Profile table, including their data types, keys, and a description of each attribute.

Field Name	Data Type	Key	Description
UserID	Integer	Primary / Foreign	Unique identifier for each profile. References User(UserID).
Bio	Text		Short user description, e.g. training goals or experience.

TrainingPreferences	Varchar(100)		Preferences such as sparring intensity or training focus.
Availability	Varchar(100)		Days or times the user is available to train.
PreferredDistance	Integer		Optional radius (in km) for searching partners and services.

One-to-One Relationship with the User Table

The relation between User and User_Profile is one-to-one. It is due to:

- Each user is allowed to have one profile record only.
- Each profile record is associated with exactly one user
- This relationship is implemented by using the UserID as both:
- The primary key for User_Profile

The foreign key referencing User(UserID). This type of design enforces referential integrity because a profile cannot exist unless a user exists. Additionally, this design ensures that more than one profile for a single user cannot be created since the UserID has to be unique in the User_Profile table.

Purpose of the Service Table

The rationale for maintaining a Service table is that users may need to access information relating to organisations and people, and BrawlBase. This includes gyms, coaches and medical professionals such as physiotherapists and sports therapists. The questionnaire results highlighted that it is difficult to find these services in one place and existing platforms do not meet the end user's needs. A dedicated services table supports the platform's aim of centralising these resources. Rather than creating completely separate tables for Gym, Coach and MedicalProfessional, a single Service table is used to store shared attributes. This reduces duplication and simplifies searching and filtering, because users can query one table and filter by service type.

Service Table Attributes

The table below outlines the fields stored in the Service table, including data types, keys, and descriptions.

Field Name	Data Type	Key	Description
ServiceID	Integer	Primary	Unique identifier for each service listing.
ServiceType	Varchar(20)		Identifies the type of service, e.g. Gym, Coach, Medical.
Name	Varchar(80)		Name of the gym, coach, or medical professional.
Location	Varchar(60)		City or area where the service is based. Used for filtering.
MartialArtSpecialism	Varchar(40)		Discipline the service supports, e.g. MMA, Boxing, BJJ.
Description	Text		Summary of the service, experience, or facilities.
ContactMethod	Varchar(80)		Contact options such as email, phone, or website.
Verified	Boolean		Indicates whether the service has been verified by the system admin.

Justification for Separating User_Profile from User

Although it's possible to store profile information in the User table, this would create several problems:

- Many users would not want to fill in optional information, and thus many records would contain NULL values.
- If all information were to be stored within the Users table, it would lead to its size becoming unnecessarily big, hence affecting efficiency on every other frequent operation, like login or search.
- The profile fields will most likely be extended in further development, thus increasing the complexity of the core User table.

By separating the information of the profile into User_Profile, the database will be more efficient and easier to maintain. Further, the fact that optional information is stored separately contributes to scalability: every new profile field can be added without affecting the creation and authentication logic of the account.

Link to Requirements and Success Criteria

The User_Profile table assists in the requirements and success factors where improvement of accuracy in matching and searching is needed, including:

- providing more detailed profile information for training partner selection
- enabling users to specify their availability and preferences to have better matching results
- enhancing usability via offering means of editing profiles without changing login details themselves

This fulfils the need for the user as outlined within the questionnaire responses, in that the end user indicated that locating appropriate training partners and support services is difficult and needs to be centralised with personalised filtering based upon personal data obtained about the individual.

Referential Integrity Considerations

In order to maintain referential integrity, User_Profile.UserID must reference an existing User.UserID. This ensures that:

- A profile cannot be created for a nonexistent account
- A user cannot have multiple profiles
- If the user is removed, the profile information should be deletable so that no orphaned data exists

This ensures data consistency while confirming that data stored in the profile is tied to a valid user account.

UserServiceInteraction Table Design

Purpose of the UserServiceInteraction Table

The purpose of the UserServiceInteraction table is to manage users who are interacting with different services on BrawlBase. This is because a user may have the ability to interact with various services, and on the other hand, a service may have the ability to be interacted with by numerous users, therefore creating a many-to-many relationship between users and services, and the UserServiceInteraction table does indeed resolve this relationship in an efficient manner.

This table enables tracking of an action such as saving a service, requesting contact, or initiating communication with a gym, a coach, or a medical professional. By storing the data in this manner, independent from other services' data, duplicate data from services is avoided. Accordingly, without altering any tables in User or Service, tracking history can be accomplished.

UserServiceInteraction Table Attributes

The table below shows the fields stored in the UserServiceInteraction table, including their data types, keys, and descriptions.

Field Name	Data Type	Key	Description
InteractionID	Integer	Primary	Unique identifier for each user-service interaction.
UserID	Integer	Foreign	References User(UserID). Identifies the user involved in the interaction.
ServiceID	Integer	Foreign	References Service(ServiceID). Identifies the service involved.
InteractionType	Varchar(30)		Describes the type of interaction, e.g. "Saved", "ContactRequested", "Viewed".

Timestamp	DateTime		Records when the interaction occurred.
Status	Varchar(20)		An optional field indicating the state of the interaction (e.g. pending, completed).

Resolution of the Many-to-Many Relationship

The relationship between users and services is inherently many-to-many:

- One user can use more than one service.
- A single service can be used by many users.

The problem is solved within UserServiceInteraction, as one record exists for every interaction. It references exactly one user and one service, so all relationships can be tracked. It is a relational database design, avoiding duplication of information within the User table or services within the UserService table.

Primary and Foreign Key Design

The InteractionID field is used as the primary key to uniquely identify each interaction. This will allow, if needed, multiple interactions between the same user and service to be recorded, like any repeated contact requests or future interaction types. User ID and Service ID are foreign keys for the User and Service tables, respectively. These foreign keys would guarantee referential integrity in that:

- An interaction can't exist if the user it references doesn't exist
- An interaction cannot exist independent of the referenced service's existence.

Constraints and Validation Considerations

To ensure data consistency and to exclude invalid data, the following constraints will be applied:

- The UserID and ServiceID cannot be NULL because the interaction between these two entities cannot happen in isolation
- "InteractionType" should be validated, comparing it with specific values defined earlier
- Timestamp is automatically generated during the creation process of the interaction.
- Status values are limited to a predefined list if used
- These constraints guarantee that interaction data will continue to be significant.
- To maintain data consistency and prevent invalid data, the following constraints will be applied:

- UserID and ServiceID cannot be NULL because any interaction must require a combination of these two entities.
- InteractionType shall be validated to match the predefined values.
- Timestamp is generated automatically when the interaction is created

Values of status are restricted to a certain set if used. The restrictions help guarantee that data on interaction is meaningful and credible.

Link to Requirements and Success Criteria

The UserServiceInteraction table directly facilitates several functional requirements and success factors, such as:

- Allowing them to interact with gyms, coaches, and medical personnel
- Providing for contact and interaction without revealing personal contact details
- Supporting potential future features like recommendations and reviews

This design partly meets the need for the user as presented within the questionnaire results, whereby the end user was facing difficulties concerning being able to access relevant services in one place.

Referential Integrity and System Reliability

The referential integrity is maintained using foreign key constraints between UserServiceInteraction and User, as well as the Service tables. This ensures that interaction data is always valid, thereby avoiding orphans, which could otherwise cause search results and functionality to malfunction or produce incorrect results. When user service relationships are kept separate from each other in a junction table, it becomes easy to maintain the system.

Algorithm Design Overview

Algorithms are the logical basis for the BrawlBase system, and they indicate how data is processed in order to provide some crucial features. The rationale behind designing algorithms at this level is that it provides a plan of the logic of the system prior to its implementation, ensuring that a process in the system is as efficient, effective, and relevant as possible.

Also, each algorithm has been designed in such a manner that the logic of the code follows a structured methodology that makes the code easy to understand. The consideration has been made in such a manner that the code can effectively execute using a high-level programming language like Python.

Input-Process-Output Structure.

All algorithms in BrawlBase follow the Input–Process–Output (IPO) model. It will be very clear what data enters into an algorithm, what processing happens, and what results leave the algorithm. Using an IPO approach will improve readability and make it clear where any validation of data and error checking needs to be developed.

Data is input or fetched from the database, depending on the user, in any manner. For example, it could contain a login information, or a searching criteria or the message.

Process: A set of ordered steps that could enable input validation, database querying, application conditions, or filters.

Output: The results are returned to the user, such as confirmation messages, search results, etc.

Additionally, this structured methodology also helps with testing because individual portions of the algorithm can be checked.

Validation and Error Handling

While SQL Server

Validation tops the list in terms of consideration while designing all algorithms. In this process, the validation of the data takes place before actually processing the data, so that improper or False data does not enter the system. For example, login algorithms perform validation in terms of login IDs, and search algorithms perform validation in terms of search values. Error handling has been implemented with the intention of offering clear and friendly guidance without giving out critical information about the system. For example, instead of directly revealing if there is an existing username in the database upon failure of a login process, an error message is passed instead.

Efficiency and Scalability Considerations

The algorithms have been developed to strike a balance between simplicity and efficiency. The database queries are written such that they process the query in an efficient manner without any unnecessary processing or redundant data handling. For example, in the algorithms for searching, the data obtained from the search process is usually filtered using a set of desired criteria instead of being loaded into memory for filtering.

Scalability has also been a concern. In fact, as the number of users, services, and messages continues to rise, the performance of the algorithms should not be compromised. The development of an algorithm based on indexed fields, such as the primary key and frequently used fields, will be helpful.

Link to Requirements and Success Criteria

Each algorithm is built to meet functional needs as well as success criteria defined in the analysis stage. For instance:

- Authentication algorithms, user registration criteria for successful login
- Search algorithms support training partner and service discovery requirements
- Messaging algorithms support secure user communication

By clearly linking algorithms to requirements, the design demonstrates that system functionality is intentional and user-driven rather than arbitrary.

Preparation for Implementation

The programming algorithms presented in the next sections of this chapter are represented in structured pseudocode. Pseudocode is a programming notation where the programming logic is represented clearly. This makes the design easier to understand for the programmer. The logic is represented clearly in such a way that it can be easily translated into programming code in the implementation phase.

Each section that follows will elaborate on all the important algorithms, including inputs, steps involved, and outputs for each algorithm, along with rules/validations applied to them.

Design Traceability and Justification

Design Constraints

The design of BrawlBase was heavily influenced by various constraints that had to be taken into account in order to make the system realistic, feasible, and appropriate in its scale to an A-Level Computer Science NEA.

Technical Constraints

The system was designed with a specific architecture that enables its implementation using a web-based stack with a relational database. As such, specific features such as video analysis in real time, AI-based feedback systems, or even data analysis were intentionally left out of the system design. This is because it would require implementation technologies that are beyond the scope of this project. In fact, these features could compromise reliability and efficiency.

Additionally, the database design had to be "manageable" while at the same time evidencing clear use of relational concepts like "primary key," "foreign key," "junction table," etc. This limited the number of tables and relations that were practicably feasible to program and test within the time available.

Time Constraints

The project is subject to strict deadlines, meaning the design had to prioritise core functionality over optional or advanced features. Features such as secure user authentication, searching for training partners and services, and messaging were prioritised because they directly address the main problem identified in the analysis phase.

More complex features, such as automated recommendations based on long-term training history or advanced moderation tools, were considered but excluded to ensure the system could be fully designed, implemented, tested, and evaluated within the available timeframe.

Scope Constraints

To prevent the project from becoming overly complex, BrawlBase was designed as a centralised discovery and communication platform, rather than a full social media or training analytics system. This, for instance, involves the ability for users to message one another, but it excludes group chats, file sharing, or public forums in an attempt to keep scope focused and manageable.

These constraints ensured the final design remained achievable while continuing to meet all key user needs identified during analysis.

Design Decisions and Trade-offs

There were several important decisions during this design process, especially during intervals involving trade-offs. One such decision was the application of a single Service table for the specification of all gyms, coaches, and medical professionals. Another option was to apply separate tables for separate kinds of service. However, it could have resulted in more problems of redundancy as well as a more complicated search option.

Using a unified table with a single attribute for ServiceType is more scalable with more intricate validation.

Another decision in design was to have two tables, User and User_Profile. Combining these two into one table would have made it easier to query them, but then NULL values would have accumulated, thus reducing normalisation.

For messages, each message is represented as a separate entity instead of using a simple junction table. This approach also gives room for additional information, such as time stamps and status. It can also, in the future, support features such as read receipts for messages. It, however, results in additional storage usage, which is considered worthwhile.

Overall, the chosen design prioritises clarity, maintainability, and data integrity over minimal table count or overly complex optimisation.

Design Traceability Matrix

The table below demonstrates how requirements identified in the analysis phase are directly supported by specific design elements and algorithms. This ensures that every major design decision is purposeful and traceable.

Requirement / Success Criteria	Design Component	Algorithm(s)
--------------------------------	------------------	--------------

Users must be able to register and log in securely	User table, PasswordHash field	Registration Algorithm, Login Algorithm
Users must be able to find training partners by skill, martial art and location	User, User_Profile tables	Training Partner Search Algorithm
Users must be able to find gyms, coaches and medical professionals	Service table, UserServiceInteraction table	Services Search Algorithm
Users must be able to communicate securely	Message table	Messaging Algorithm
Data must remain consistent and valid	Primary/Foreign keys, junction tables	All database-related algorithms

This traceability confirms that the system design directly addresses the problems identified in the analysis phase and that no features have been included without justification.

Design Summary

The design phase for BrawlBase has given way to a full and structured blueprint of implementation. A relational database has been closely designed to accommodate user accounts, services, interactions, and messaging while supporting normalisation and referential integrity. Where necessary, junction tables have been used to resolve many-to-many relationships, thereby allowing scalability and data consistency. The designs of algorithms have also been proposed, keeping in mind the structured approach toward programming, based on the Input-Process-Output model. Besides, the proposed design allows handling user interfacing, security, and realistic project boundaries to implement the system within a feasible time and technical limits.

Overall, the proposed design fully meets the required functional and non-functional requirements specified in the analysis phase and paves the groundwork for the implementation stage. The design stage of BrawlBase has presented a comprehensive blueprint of implementation. For instance, a database design with relation concepts such as normalisation and referential integrity has been implemented with regard to service, user, interaction, and messaging systems. Junction tables have been implemented as required in many situations. The algorithms have been designed with a structured methodology and use an Input/Process/Output modelling technique with any required validation details. This design also meets all requirements for usability, security, and project constraints normally imposed, making it practical within an ascertainable time and current technological capabilities. The design overall satisfies all the functional and non-functional requirements defined in the analysis phase, thus providing a good background for the implementation stage.

Design Decisions and Trade-offs

Throughout the design process for the development of BrawlBase, several design decisions have been made, which have been important for the development of the system with respect to different alternatives. This is important considering the need to meet the user requirements identified during the analysis phase. Some of the key decisions that had to be made were whether or not a relational database style would need to be employed, as opposed to a non-relational database, also known as a NoSQL database, as it could be argued that it might offer more flexibility for dealing with unstructured data, even if BrawlBase does employ structured relationships. The use of a relational database approach allows these structured relationships to be defined more naturally. Another critical decision taken in this regard was related to the segregation of the User and User_Profile tables. Although it would have made things easier if all the data related to the users had been placed within the same table for easier query writing, NULL values are avoided in database normalisation. By separating core authentication data from optional profile information, the database becomes more efficient and easier to extend in the future. The trade-off is the need for additional joins when retrieving full profile data, which was considered acceptable given the benefits to data organisation and scalability.

A further design decision was the use of a single Service table to represent gyms, coaches, and medical professionals. Another option could have been to design a separate table for each service type. Although this may have minimised validation on the ServiceType field, it would have greatly complicated searching and filtering services. This would have led to a simple design while accessing services; however, validation may get complicated.

Lastly, the messaging system was also considered. Under this, each message was implemented as a single entity rather than following the simple approach of creating a junction table between the user tables. This has provided a scope for the incorporation of additional information, including status and timestamps. However, this has also meant a greater storage requirement. Based on the project requirements, this was considered to be acceptable as the reliability of communication in the application is paramount. Finally, decisions were made regarding limiting the scope of advanced features. For instance, although further investigation highlighted a potential interest in providing personalised feedback and analysis for training data, advanced analysis tools or media tools would greatly increase the complexity and risk involved with these tools. The advanced features will not be included in this basic system design.

Overall, the decisions made for the design of BrawlBase favour clarity, reliability, and maintainability over complexity. Each of these trade-offs has been analysed in such an approach, which ensures that, at the end, the design of the system should be one that is achievable, user-friendly, and suitable for an A-Level Computer Science NEA. The decisions reached regarding the BrawlBase design all culminate in an implication of an emerging product where simplicity, reliability, and ease of maintenance are deemed crucial over other complicated elements or features.

Design Traceability Matrix

The reason for creating this traceability matrix is to prove that all important design decisions regarding BrawlBase are traced to a requirement/success criterion that was identified in the analysis phase. This way, it is ensured that all design decisions for the system are meaningful and that there are no components that have gone into the system without a reason. For instance, the matrix below shows how each requirement is supported by particular database structures, algorithms, and interface designs.

Design Traceability Matrix Table

Requirement / Success Criteria	Design Component(s)	Algorithm(s)	User Interface Element(s)	Test Coverage
Users must be able to register securely	User table, PasswordHash field	User Registration Algorithm	Registration page	Valid registration test, duplicate username test, password length boundary test
Users must be able to log in with valid credentials	User table	Login Algorithm	Login page	Valid login test, incorrect password test
Users must be prevented from accessing accounts with incorrect details	Validation rules, error handling strategy	Login Algorithm	Login page error messages	Erroneous login test
Users must be able to search for training partners by skill level, martial art, and location	User, User_Profile tables	Training Partner Search Algorithm	Search partners page	Search by skill level test, invalid filter test

Users must be able to find gyms, coaches, and medical professionals	Service table	Services Search Algorithm	Services directory page	Search by service type test
Users must be able to interact with services	UserServiceInteraction table	Services Search Algorithm	Service profile page	Interaction creation test
Users must be able to communicate securely	Message table	Messaging Algorithm	Messaging page	Message send test, invalid recipient test
Data relationships must remain valid	Primary keys, foreign keys, junction tables	All database-related algorithms	N/A	Referential integrity tests
The system must be usable and accessible	UI design principles	N/A	All pages	Usability testing
The system must be secure	Password hashing, validation rules	Registration and Login Algorithms	Login and registration pages	Security-related tests

Explanation of Traceability

This matrix also indicates that all functional requirements or success criteria identified within the analysis phase are being met at least by one of the components identified in the design phase. For example, the functional requirement of having a secure login system will be satisfied by the design of the user table, password hashing, and login algorithms, along with the targeted testing performed. In a way, the traceability matrix provides clear visibility that the relationship between the requirements and the decisions made in the algorithm can be directly related to a methodical and structured implementation of the system. It also

promotes a methodical way in the execution of the tests since each requirement comes with a group of tests.

Importance of Traceability

Including a traceability matrix strengthens the overall design by:

- ensuring no requirements are overlooked
- preventing unnecessary features from being included
- supporting structured testing and evaluation
- demonstrating clear planning and justification to the examiner

This confirms that BrawlBase has been designed to directly address the problems identified in the analysis phase and is fully prepared for the implementation stage.

BrawlBase - Language and IDE Choice

Language and IDE Choice

Python is the selected language

Python has been selected as the programming language for BrawlBase. Python has an enormous standard library as well as quite a big selection of third-party modules which can be used for networking, data storage and user interface. Python's accessibility in being able to change it however you like means it can be rapidly used to develop prototypes whilst the project is ongoing.

The second reason for choosing Python is familiarity through personal experience. Everything has been programmed in Python previously, and hence there is already a working familiarity with its syntax, debugging environment and library ecosystem. This allows for writing good and clean code without taking too much time learning new structures. Other languages such as Java or C++ were also taken into consideration, but they are more complex setup and take more development time, which is not optimal for this project.

Advantages of Python for BrawlBase

- Simple syntax that is novice user-friendly and reduces errors
- Rich ecosystem of modules to enable database connections, security and user interfaces
- Strong support for test-driven development via built-in frameworks
- Interpretive nature means immediate feedback when testing
- Huge community support, with the assumption being that solutions and tutorials are easily accessible

Why was Visual Studio Code used?

The Development Environment applied here in this project is Visual Studio Code. It has been selected from experience as well as appropriateness for development in Python. Visual Studio Code is light on hardware and operates on all the leading platforms (Windows, in my case), thus it is flexible for development on devices. It supports extensions that have choices such as Python debugging, linting, Git integration and virtual environment management.

Having already done projects before with Visual Studio Code, there is a head start in knowing how it looks, shortcuts and what tools are available. This reduces setup time, and focus can be brought to building the system itself. Other IDEs such as PyCharm, Eclipse and IntelliJ were tested, but they all had flaws such as being heavy on resources, requiring a paid version, or being designed for a language other than Python.

Advantages of Visual Studio Code

- Open source and free
- Customizable with extensions
- Terminal built-in to execute and test the code quickly
- Git integration for version control
- Strong community support with frequent updates

This combination of Python and Visual Studio Code will make a good combination when it comes to the development of BrawlBase.

BrawlBase – Summary and Next Steps

It has also been noted that a significant outcome of the design phase involved the formulation of an integral and organised blueprint concerning the implementation phase. Every decision made at the design phase has been made with specific reference to the needs that were identified, as mentioned at the analysis phase. The relational database design was adopted to address the structured needs of data as well as their complex relationship about users, services, and interactions. To ensure proper data integrity through

association, primary keys, foreign keys, and junction tables were implemented to address many-to-many relationships appropriately. Also, data normalisation was ensured to avoid redundancy and enhance data consistency without compromising scalability.

The algorithms have also been designed with a well-defined structure, i.e., Input, Process, Output, which ensures that due consideration has been given to matters of validation as well as possible errors, keeping in view the logic skeleton for the actual programming of the main system functions like user registration, login, search, transactions, etc. This allows the actual programming to be done most efficiently with minimum logical programming errors. User interface designs were created with usability, accessibility, and consistency in mind. A mobile-first approach was adopted to reflect how users are most likely to access the platform. The wireframes and other layout decisions are based on the feedback from the questionnaire, which ensures that the interface allows for quick and instinctive navigation while avoiding unnecessary added complexity.

Considerations related to security and validation have been addressed throughout the design process. Password hashing, validation, and prevention of common attacks have been planned to ensure proper security and usability of user data, as well as correct system behaviour under normal as well as abnormal cases. Finally, a design traceability matrix was provided to demonstrate the existence of a link between the requirements, design components, algorithms, and testing. This ensures that all the components included in the design have a definite purpose towards fulfilling the objectives of the system.

Thus, it could be concluded that the design phase has provided a coherent and well-grounded system design, as all the requirements have been met by the system, which are realistic considering several constraints with respect to the project. Thus, the system is now fully ready to move on with respect to the implementation phase.

Design Phase Evaluation – Dashboard and Wireframes

The dashboard was created with the intention of designing the user interface with wireframes during the design phase. The wireframe had all the essential elements of the dashboard, such as a navigation bar, links to other features such as search and messaging, and a section to hold user-specific data. This helped to visualize the user interface before development.

The final version of the dashboard looks similar to the wireframe version created during the design phase. The structure of the dashboard, including the navigation bar and placement of all the essential elements, is similar to the original version. This indicates that the design phase of the development life cycle was successful in designing the user interface of the system. The system does not need to make significant changes to the structure of the user interface during the implementation phase.

Wireframe:

```

-----
|   Welcome, Ahmed   |
|-----|

```

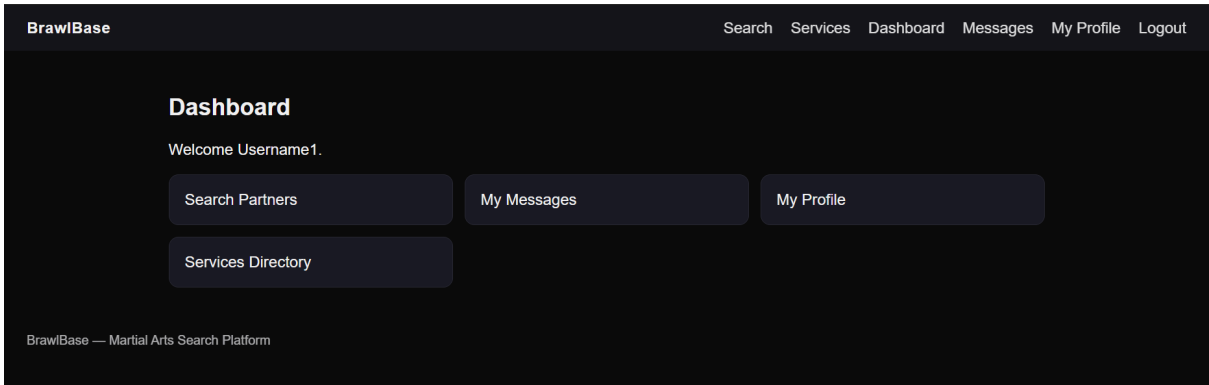
| [Search Partners] |

| [My Messages] |

| [My Profile] |

| [Services Directory] |

Website Dashboard



Strengths	Limitations
The wireframe clearly planned the layout, navigation and structure of the dashboard before implementation began.	The wireframe only provided a basic layout and did not fully represent styling or visual design elements.
Key components such as the navigation bar, search and messaging links were included, making the system easy to plan and develop.	The wireframe did not account for user experience details such as spacing, alignment and overall aesthetics.
The final dashboard closely followed the original wireframe, showing that the design phase was effective and accurate.	Some elements had to be repositioned during development, meaning the wireframe was not perfectly optimised initially.

The design helped maintain consistency throughout development and reduced the need for major changes.	The wireframe did not fully show how dynamic data (e.g. user-specific information) would be displayed.
It provided a strong foundation that made the implementation process more organised and efficient.	Additional adjustments were required during implementation to improve usability and presentation.

BrawlBase – Coding The Website (Technical Solutions)

Services Data Models – models.py

```

5  db = SQLAlchemy()
6
7  class User(db.Model):
8      __tablename__ = "users"
9      id = db.Column(db.Integer, primary_key=True)           # UserID
10     username = db.Column(db.String(50), unique=True, nullable=False)
11     password_hash = db.Column(db.String(255), nullable=False)
12     martial_art = db.Column(db.String(50), nullable=True)
13     skill_level = db.Column(db.String(30), nullable=True)
14     location = db.Column(db.String(80), nullable=True)
15
16     def set_password(self, raw_password: str) -> None:
17         self.password_hash = generate_password_hash(raw_password)
18
19     def check_password(self, raw_password: str) -> bool:
20         return check_password_hash(self.password_hash, raw_password)
21
22 class Gym(db.Model):
23     __tablename__ = "gyms"
24     id = db.Column(db.Integer, primary_key=True)           # GymID
25     name = db.Column(db.String(80), nullable=False)
26     location = db.Column(db.String(80), nullable=True)
27     facilities = db.Column(db.Text, nullable=True)
28
29 class Coach(db.Model):
30     __tablename__ = "coaches"
31     id = db.Column(db.Integer, primary_key=True)           # CoachID
32     name = db.Column(db.String(80), nullable=False)
33     specialism = db.Column(db.String(80), nullable=True)
34     location = db.Column(db.String(80), nullable=True)
35
36 class Professional(db.Model):
37     __tablename__ = "professionals"
38     id = db.Column(db.Integer, primary_key=True)           # ProfID
39     name = db.Column(db.String(80), nullable=False)
40     profession = db.Column(db.String(80), nullable=True)
41     location = db.Column(db.String(80), nullable=True)
42
43 class Message(db.Model):
44     __tablename__ = "messages"
45     id = db.Column(db.Integer, primary_key=True)
46     sender_username = db.Column(db.String(50), nullable=False)
47     receiver_username = db.Column(db.String(50), nullable=False)
48     content = db.Column(db.Text, nullable=False)
49     timestamp = db.Column(db.DateTime, default=datetime.utcnow)
50

```

The file above creates the primary data structures that are used in the BrawlBase system. Each of the data structures defined in the file represents a table in the database. The data structures are the backbone of the data management system. The use of SQLAlchemy allows the data structures to be defined as Python code and still utilize a relational database.

The User data structure is used for the management of account information. The data structure has unique identifiers for each user. The identifiers include the martial art practiced by the user and the skill level. The username is also unique for each user. This prevents the creation of duplicate accounts. It also allows for the proper identification of each user. Passwords are not stored as plain text. Instead, the hash of the password is stored. This

hash is created by a hashing function. The hashing function is a secure way of storing passwords. The `set_password` and `check_password` functions are used for this purpose.

The Gym, Coach, and Professional models are used to store various types of services available on the platform. Each model has a separate table to store this data, which remains well-organized. This way, each category has a few unique attributes as well. For example, coaches have a `specialism` field, while professionals have a `profession` field. The Message model is used to store messages between users. Each message has a sender username, a receiver username, content, and a timestamp. The timestamp is automatically generated upon creation of each message, which allows them to be displayed in chronological order. Storing each message in a separate table allows this data to remain separate from user account information, making this database more structured.

From a design standpoint, it is evident that each model uses a primary key identifier labeled "id." This is important from a database standpoint and allows records to be identified and referenced accordingly. The use of nullable fields for items such as location and skill level allows for a greater degree of flexibility in that users are not forced to provide this information when creating a user account. The design itself follows a relational database model in that different types of data are segmented into distinct models. This allows for a greater degree of efficiency in that queries may be made without introducing undue complexity into the system. It also lends itself well to scalability in that models may be added without impacting existing functionality. Finally, it is worth noting that the use of SQLAlchemy allows for a degree of abstraction between the application and database itself. This means that in theory, a different database system may be used in the future without requiring significant alterations to the existing codebase.

This file is important in that it ensures that all data in the system is stored and accessed in a particular way. It supports some important functions such as authentication, search, messaging, and services. It is therefore a very important file in this application.

Messaging Data Model – models.py

```

43 class Message(db.Model):
44     __tablename__ = "messages"
45     id = db.Column(db.Integer, primary_key=True)
46     sender_username = db.Column(db.String(50), nullable=False)
47     receiver_username = db.Column(db.String(50), nullable=False)
48     content = db.Column(db.String(500), nullable=False)
49     timestamp = db.Column(db.DateTime, default=datetime.utcnow)
50

```

This class is utilised for storing messages that are exchanged between users within the system. Each message is stored as a separate record in the database to enable conversations to be stored for future reference. Every message has a unique ID, which allows the system to track individual messages and display them correctly in inboxes and sent message views. This also prevents messages from being overwritten or confused with one another. The sender and receiver are stored using usernames rather than numeric IDs. This makes the message system easier to work with when displaying conversations, as usernames are already used throughout the interface. It also reduces the need for additional joins when retrieving messages.

The message content is stored as text and is required. This ensures that the user cannot send a blank message, making the service more user-friendly by preventing the database from accumulating unnecessary information. A timestamp will be automatically included in the message when the message is sent. Using the timestamp, messages will be in order, reflecting the most recent messages. This will also enable the user to know the timing of the messages. By storing the messages in their own table, the system is thus keeping the communication information distinct from the information in the user accounts. By doing so, the database becomes more organised and easier to handle. The messaging model supports both inbox and sent message views by allowing messages to be filtered based on sender and receiver. This allows conversations from multiple perspectives to be demonstrated in a way that avoids duplicating the data.

A database-driven messaging system ensures that the messages remain even when the server has restarted. This is significant for reliability purposes, meaning that the users will not lose their communication history. Comments can be used for commenting key fields, such as the message ID. This makes it more understandable to read through. Such a model will help facilitate the connect and messaging functionality offered in the platform since it will enable users to communicate safely.

Register Page Form (register.html)

```

1  {% extends "base.html" %}
2  {% block content %}
3      <h2>Register</h2>
4      <form method="post" class="form">
5          <label>Username</label>
6          <input name="username" required>
7
8          <label>Password (min 6)</label>
9          <input name="password" type="password" required>
10
11         <label>Martial Art</label>
12         <input name="martial_art" placeholder="Boxing, MMA, Kickboxing">
13
14         <label>Skill Level</label>
15         <input name="skill_level" placeholder="Beginner, Intermediate, Advanced">
16
17         <label>Location</label>
18         <input name="location" placeholder="City or postcode">
19
20         <button type="submit">Create account</button>
21     </form>
22 {% endblock %}
23

```

This is the initial phase of the backend operations, which consists of checking, for instance, if the password entered is of the right length, if the username given is not used by someone else, and then making the password unrecognisable with a hash function before it gets stored. In this manner, the HTML remained extremely straightforward, with the server looking after the validations and security-related processes. This not only neatly organises the code but also helps in avoiding the critical task of executing the sensitive operations outright in the HTML code. This file is responsible for the creation of the registration page, where users can register and become part of the system. It has an HTML form that takes the basic details required for the creation of an account. It has been created using the POST method, meaning that the details provided by the user will not be visible in the URL, as they will be transmitted in an encoded manner. The form consists of fields for the username and password. Both fields have been marked as 'required.' This ensures the form is not submitted with the fields being empty. This goes a long way in ensuring there are fewer basic errors at the point the information is submitted for processing. The type of the password field is set to "password," so the characters typed get concealed as the user is typing. This is significant since it ensures the password is not visible to others, especially when the user is creating the account in a public environment. Additionally, this ensures the system feels secure. Extra fields are included for martial arts, skill level and location. These fields are used to store profile information that can later be used for searching and matching users. For example, users can search for others based on martial arts or location. These fields are optional because not all users will want to provide this information straight away. Placeholder text is used inside several input fields to show examples of what the user should enter. This helps guide the user and makes the form easier to understand without needing extra instructions on the page. It also reduces the chance of users entering incorrect or unclear information. The submit button at the bottom of the form allows the user to send their details once they are finished. When the button is clicked, we submit our form data through

the register route function on the backend. This begins the registration process on the server-side, involving the checking of whether the password submitted is of the correct size and whether the submitted name is unique, and subsequently applying the hash function on the password.

In this way, the HTML code stays simple, and the validations and security-related functionalities are taken care of on the server-side. This helps in managing the code efficiently and prevents the critical task of performing the sensitive operations directly within the HTML code. Essentially, this form is the starting point within the account creation form and is a key component within the system that provides a simple way to input information that supports all the functionality within the system, such as searching and login functionality.

Register Route Backend Logic (app.py)

```

52 @app.route("/register", methods=["GET", "POST"])
53 def register():
54     if request.method == "POST":
55         username = request.form.get("username", "").strip()
56         password = request.form.get("password", "")
57         martial_art = request.form.get("martial_art", "").strip()
58         skill_level = request.form.get("skill_level", "").strip()
59         location = request.form.get("location", "").strip()
60
61         if len(username) < 3:
62             flash("Username must be at least 3 characters.")
63             return render_template("register.html", user=current_user())
64         if len(password) < 6:
65             flash("Password must be at least 6 characters.")
66             return render_template("register.html", user=current_user())
67         if User.query.filter_by(username=username).first():
68             flash("Username already exists.")
69             return render_template("register.html", user=current_user())
70
71         u = User(
72             username=username,
73             martial_art=martial_art,
74             skill_level=skill_level,
75             location=location
76         )

```

This is the function that manages the action performed when the user submits the registration form. This function deals with the loading and processing of the submitted information by the user. The function responds to the GET and POST request methods. This allows the same function to handle the display and processing of the form. When the request method is POST, the system knows that the user has submitted the form. The values entered into the form are then retrieved using `request.form.get`. Each input is collected separately, including username, password, martial art, skill level, and location. The `strip()` method is used on text inputs to remove any unnecessary spaces at the beginning or end of the input.

Before creating a new account, several validation checks are carried out. First, the username length is checked to make sure it is at least three characters long. This prevents very short or unclear usernames from being used. If the condition is not met, an error message is shown, and the registration page is reloaded. The password length is also verified to ensure that it is a minimum of six characters long. This is done to ensure that passwords that contain very little strength are not employed. If the password fails the length requirement, the user is notified and asked to retry.

The next check performed by the system is whether the username already exists in the database. This is accomplished by searching the User table within the database for the same username. When a user with that username is found, the registration procedure ends with an error message such as "This user is already taken. If all validation checks pass, a new User object is created using the data provided. The extra fields for the profile, such as

martial art, skill level, and location, are added at this level to ensure they are recorded together with the account information. The password itself is not stored directly. Its storage is done after the password is submitted to another function, where the password is hashed before the storage of the password within the database. With the user object now ready, it can then be added to the database session. This permanently saves the new account details in the database. The user is then able to log in using the details they provided. Throughout this process, error messages are used to give clear feedback to the user if something goes wrong. This makes the system easier to use and helps guide users through the registration process. In this manner, this approach improves the clarity and functionality of the code.

Admin Services Route – Managing Platform Data

```

137 @app.route("/admin/services", methods=["GET", "POST"])
138 def admin_services():
139     if not login_required():
140         return redirect(url_for("login"))
141
142     if request.method == "POST":
143         form_type = request.form.get("form_type")
144
145         if form_type == "gym":
146             db.session.add(Gym(
147                 name=request.form.get("gym_name", "").strip(),
148                 location=request.form.get("gym_location", "").strip(),
149                 facilities=request.form.get("gym_facilities", "").strip()
150             ))
151
152         elif form_type == "coach":
153             db.session.add(Coach(
154                 name=request.form.get("coach_name", "").strip(),
155                 specialism=request.form.get("coach_specialism", "").strip(),
156                 location=request.form.get("coach_location", "").strip()
157             ))
158
159         elif form_type == "professional":
160             db.session.add(Professional(
161                 name=request.form.get("pro_name", "").strip(),
162                 profession=request.form.get("pro_profession", "").strip(),
163                 location=request.form.get("pro_location", "").strip()
164             ))
165
166         elif form_type == "seed":
167             db.session.add(Gym(name="BrawlBase Gym", location="London", facilities="Ring"))
168             db.session.add(Coach(name="Coach Saad", specialism="MMA", location="Manchester"))
169             db.session.add(Professional(name="Dr Ali", profession="Physio", location="Leeds"))
170
171         db.session.commit()
172         return redirect(url_for("admin_services"))
173
174     return render_template("admin_services.html", user=current_user())
175

```


This piece of code contains functionality for the admin services page within my project. Primarily, the code ensures that authorised individuals are able to submit various services, such as gyms, coaches and professionals, to the database. This piece of functionality is essential because BrawlBase will essentially allow individuals to connect to training locations as well as professionals. Therefore, there must be an organised means by which this information can be stored. The first thing that the route does is verify whether the user is logged in through the use of the `login_required()` function. If the user does not log in, they are immediately redirected to the log-in page. This adds a basic layer of security and prevents unauthorised users from accessing admin-only functionality. Without this check, anyone could potentially add or modify services in the database, which would make the system unreliable. When the form on the admin services page is submitted, the program checks if the request method is POST. This means it can use the same route to display the page as well as handle the form submissions. Something like this just really cleans up the code a bit and prevents duplication. It then takes a variable called `form_type` from the submitted form. This informs the program what type of data the user is attempting to implement.

Each kind of form is treated with conditional logic. If the type is "gym", it instantiates a new Gym class with the user-entered values. It retrieves the values from the form and cleans them using the `.strip()` method, which removes leading and trailing spaces. Thus, this prevents inconsistent and messy data in your database. This same mantra is extended towards the handling of coaches and professionals, where each choice creates an object matching the relevant table in the database. That's what the designers wanted- different routes to handle various kinds of data. Having routes only for gyms, coaches, and professionals would be more difficult to maintain in the future. For example, if another type of service needed to be added to the application in the future, it would be necessary to add only another condition block.

Once the appropriate object has been created, it is added to the SQLAlchemy session using `db.session.add()`. At this stage, the data exists temporarily in memory but has not yet been saved permanently. The call to `db.session.commit()` is essential because it writes all pending changes to the database. Without committing the session, the new records would be lost as soon as the request finishes.

After the data is saved, the user is redirected back to the admin services page. This way, the user will receive a clear notification that the operation has been completed successfully, and repeating the submission should also be avoided if the page is reloaded at the same time. The pattern of performing a POST operation followed by a POST operation redirect is quite common and should improve the experience.

It also has an option called "seed" that allows users to put predefined data into the database. The tool was applied while developing the program; it has filled the database with data from the example gyms, coaches and professionals. Including data from seed has made it simpler to test the functionality of searching data, showing data from the services, as well as connecting data from other pages.

In conclusion, this piece of code illustrates how data input, validation, and database entry about dynamic data are handled in this project using SQLAlchemy. It also illustrates how the concepts of security, maintainability, and usability have been ensured in developing this

project. With access management, logical statements, and database functionality incorporated into a single feature, this route is also a pivotal element in making BrawlBase a functional and scalable platform.

Login Page (login.html)

```
templates > <> login.html > ...
1  {% extends "base.html" %}
2  {% block content %}
3      <h2>Login</h2>
4      <form method="post" class="form">
5          <label>Username</label>
6          <input name="username" required>
7          <label>Password</label>
8          <input name="password" type="password" required>
9          <button type="submit">Login</button>
10     </form>
11 {% endblock %}
12
```

This is the login page I created for BrawlBase. Pretty much everything that's locked behind a user account — the dashboard, private messages only opens up once someone logs in successfully. Without this page, nobody who's already signed up could actually get into their own account. I decided right from the start to make it extend base.html. That was one of the better choices I made early on. The navigation bar, the header, the footer, all the global styles — they stay the same as the rest of the site. I didn't want to keep copying the same HTML blocks into every single page, and this way, if I decide later to move the logo, change colours or fix something on mobile, I only edit one file. Saves a lot of headaches. Everything that actually belongs to the login screen goes inside the {% block content %} part. Whatever I put there completely replaces the content block that's sitting in the base template. It keeps the file clean and makes it much easier to find what I'm looking for when I'm debugging. The form sends data using POST — I never even considered GET for this. Passwords in the URL would be a disaster, especially if someone glances at the screen or checks browser history. I also gave the form a class so it looks the same as the signup form and any other forms I've built.

There are just two fields: username and password. Both have proper labels so it's obvious what goes where. I added the required attribute to both inputs — nothing fancy, but it stops the form from being sent empty and gives immediate feedback before anything hits the server. For the password field, I used type="password" so the characters turn into dots as you type. Obvious, but important — especially if you're logging in from a library or shared

computer. The name attributes are username and password — exactly what my Flask route expects — so request.form picks them up without any renaming tricks. When someone hits the Login button, the data gets posted to the /login route in app.py. There, the code checks two things: does this username even exist in the database, and if it does, does the password match what's stored (hashed). If both are correct, the session is created, and they get sent straight to their dashboard. If anything's wrong, I flash a short message like "Invalid username or password" and reload the login page so they can try again. Looking back over the code, it becomes clear that the purpose of this page is to basically be the front door to the entire authentication code. It gets the details safely, passes them back to the back end, and then leaves the actual verification part to the ever-trusty Flask. Keeping the HTML and HTML with Jinja separate has made the code much easier to test and fix/expand later without breaking it.

Login Route (/login) – Backend Authentication Logic

```

39
40     @app.route("/login", methods=["GET", "POST"])
41     def login():
42         if request.method == "POST":
43             username = request.form.get("username", "").strip()
44             password = request.form.get("password", "")
45             user = User.query.filter_by(username=username).first()
46             if user and user.check_password(password):
47                 session["username"] = user.username
48                 return redirect(url_for("dashboard"))
49             flash("Invalid username or password.")
50         return render_template("login.html", user=current_user())
51

```

This section of code contains the main functionality for the login route in the BrawlBase project. Primarily, it manages the backend side of user authentication. The login.html template is where users enter their details, but this route is where the real decision happens—it checks credentials against the database and grants or denies access. And that part is very critical because without proper verification of the data, the entire security mechanism meant for securing a particular page, like the dashboard, will collapse. I specified the route as follows to ensure that both functions can be used in one app:

```
@app.route("/login", methods=["GET", "POST"])
```

So that if someone enters the page for the first time, the pages will be loaded as GET requests.

After they submit the form, it becomes POST, and that's when the verification code runs. Putting both in the same route avoids having separate endpoints for display and processing, which keeps things cleaner and less prone to errors later on. In a POST request, the code pulls the username and password straight from the form data with request.form.get(). I added .strip() to the username to catch any extra spaces someone might type by accident. Of course, that's just a small thing, but it helps prevent bad logins because of something silly like an extra space. The query to search the database uses User.query.filter_by(username =

username).first(). As user names are supposed to be unique, it will either return a user object or return none. It's a quick, targeted query that doesn't waste resources, which matters when logins happen often. If there's a matching user, the code then checks the password with `user.check_password(password)`. This method handles the secure comparison to the stored hash—I made sure from the beginning that no plain passwords ever sit in the database. That's basic security, but it's what stops a breach from exposing everything. When everything matches, login happens: `session["username"] = user.username` sets the session key. Flask sessions let the app remember the user on every page after that, so restricted areas stay locked without constant re-authentication. It's the same method that I employed throughout this project for uniform access. Immediately after this, it will redirect them to `url_for('dashboard')` through this function: `redirect(url_for('dashboard'))`. This method of redirect was used specifically to avoid the possibility of the form being resubmitted in the event of a page refresh, and it was used specifically to make it obvious that they were logged in.

If a username cannot be found or if a password doesn't match, "Invalid username or password" appears on the page. Keeping the message vague like that helps security—it doesn't tell attackers which part was wrong, so they can't easily guess or probe. For GET requests or failed attempts, the template just renders again. I pass any relevant user context to it, though usually it's just there so the user can try again immediately without getting bounced around. Thus, in conclusion, this method illustrates the right way to keep authentication secured in the project. It combines form handling, cleaning of input values, faster queries, password verification via hashing, sessions, and security in giving feedback. It does this within one endpoint, making BrawlBase more secure, easier to handle, and more reliable, with more features added.

Dashboard Route (Backend Logic)

```

33
34 @app.get("/dashboard")
35 def dashboard():
36     if not login_required():
37         return redirect(url_for("login"))
38     return render_template("dashboard.html", user=current_user())
39
40 @app.route("/login", methods=["GET", "POST"])
41 def login():
42     if request.method == "POST":
43         username = request.form.get("username", "").strip()
44         password = request.form.get("password", "")
45         user = User.query.filter_by(username=username).first()
46         if user and user.check_password(password):
47             session["username"] = user.username
48             return redirect(url_for("dashboard"))
49         flash("Invalid username or password.")
50     return render_template("login.html", user=current_user())
51

```

The code in this part is for controlling the backend of the application. It determines what happens when the user tries to enter the dashboard page. It also determines that the user has to be logged in for this page to be visible. The first part of this code, `@app.get("/dashboard")`, determines the route for the dashboard page. This means that when the user types in the browser `/dashboard`, this code will run. The use of `.get` indicates that this route is for getting information and not for sending information. This is appropriate because the dashboard page does not send information; it simply displays it. The first thing that happens in this code is the use of `login_required()`. This part of the code is important because it determines who has access. Instead of just allowing anyone to enter the dashboard page, the code checks first if the user is logged in. If the user is not logged in, the code redirects the user to the login page by using `redirect(url_for("login"))`.

This check is important because without it, someone could just type `/dashboard` into the URL and access the page without logging in. By adding this condition, the system enforces basic authentication and protects the rest of the application.

If the user passes this check, the code continues to the final line:

```
return render_template("dashboard.html", user=current_user())
```

This is the part of the code that actually loads the dashboard page. The `render_template` function is used to send the HTML file to the browser so it can be displayed. However, it also includes the `user=current_user()`, which means the details of the currently logged-in user will be passed to the template. The function `current_user()` is used to return the user that is currently logged in. This is probably done by accessing the session data that is stored when the user is logged in. For example, if a user is logging in, their username is stored in the session. This function probably uses this data to return the correct user from the database. This means that the system is able to keep track of the user that is currently accessing the system. This means that the object of the currently logged-in user is passed to the template, and it is then able to be accessed in the frontend of the website. This is how the dashboard page is able to display a personal welcome message instead of a standard one.

This is because the backend and frontend have a relationship, which shows how the data is managed. In this case, the data is not hard-coded in the HTML file, but rather real data is used from the database. To conclude, this part of the code connects all parts of the program. This is because, in this section, all parts of the program, such as routing, authentication, and showing data, are connected, thus making the program secure, dynamic, and user-specific.

Dashboard Route (Frontend Logic)

```

templates > <> dashboard.html > ...
1  {% extends "base.html" %}
2  {% block content %}
3      <h2>Dashboard</h2>
4      <p>Welcome {{ user.username }}.</p>
5      <div class="grid">
6          <a class="card" href="/search">Search Partners</a>
7          <a class="card" href="/messages">My Messages</a>
8          <a class="card" href="/profile/{{ user.username }}">My Profile</a>
9          <a class="card" href="/services">Services Directory</a>
10     </div>
11 {% endblock %}
12

```

This section of code is the template for the dashboard page. This is the page that the user will be directed to after they have successfully logged in to the system. This section is written in HTML but also utilizes the Jinja templating engine. This allows for dynamic data to be sent from the backend and rendered on the page. At the top of this section, `{% extends "base.html" %}` is used. This means that the dashboard page is using the layout for this page from another page in the system. This is useful for keeping the code organized and avoiding repetition in pages. The section `{% block content %}` is used to specify what is unique about this page. This is where the dynamic information for this page is rendered. The text `<h2>Dashboard</h2>` is simply used to display the title for this page. Below this text is a message saying "welcome" and utilizes `{{ user.username }}`, which is dynamic information sent from the backend. This means that the username for the user who is logged in is rendered on the page.

The main part of the page is inside a `<div class="grid">`. This is used to organise the layout of the dashboard into a structured format, most likely styled using CSS to display items in a grid layout. Inside this grid, there are multiple links created using `<a>` tags with the class "card". Each one acts as a navigation option for the user:

/search allows the user to search for training partners

/messages takes the user to their messages page

/profile/{{ user.username }} links to the user's own profile page

/services opens the services directory

The use of {{ user.username }} inside the profile link ensures that the correct profile is loaded based on the currently logged-in user. This avoids hardcoding and makes the system flexible for multiple users.

Overall, this page acts as a central hub for the user. It connects different parts of the system together and allows the user to easily navigate between features. It also demonstrates how frontend templates work together with backend data to create a personalised experience.

[Admin Services Page \(admin services.html\)](#)

```

templates > admin_services.html > h3
1  {% extends "base.html" %}
2  {% block content %}
3  <h2>Admin Services Directory</h2>
4
5  <p>Add gyms, coaches and professionals.</p>
6
7  <h3>Add Gym</h3>
8  <form method="post">
9      <input type="hidden" name="form_type" value="gym">
10     <input name="gym_name" placeholder="Gym name" required>
11     <input name="gym_location" placeholder="Location">
12     <input name="gym_facilities" placeholder="Facilities">
13     <button type="submit">Add Gym</button>
14 </form>
15
16 <hr>
17
18 <h3>Add Coach</h3>
19 <form method="post">
20     <input type="hidden" name="form_type" value="coach">
21     <input name="coach_name" placeholder="Coach name" required>
22     <input name="coach_specialism" placeholder="Specialism">
23     <input name="coach_location" placeholder="Location">
24     <button type="submit">Add Coach</button>
25 </form>
26
27 <hr>
28
29 <h3>Add Professional</h3>
30 <form method="post">
31     <input type="hidden" name="form_type" value="professional">
32     <input name="pro_name" placeholder="Name" required>
33     <input name="pro_profession" placeholder="Profession">
34     <input name="pro_location" placeholder="Location">
35     <button type="submit">Add Professional</button>
36 </form>

```

This file is for creating the admin services page for BrawlBase. The use of this page is for adding services into the system. This is different from other web pages, as instead of focusing on users, the system also needs to cater for gyms, coaches, and professionals. This is essentially the front end for adding that type of data into the system. This is primarily an admin area, meaning it is for the running of the website as a whole. This is as opposed to a regular user, where it is for browsing. This page is essentially an extension of the base.html. This means it looks and feels the same as any other page. I have done it this way for the entire project, as it allows me to avoid repeating the same code for the basic structure of a webpage in each individual file. This also allows for a more cohesive look, as if I ever want to change the basic structure of the website, I can do it in one place instead of multiple.

Inside the content block, I added a heading and a short line of text to make the purpose of the page obvious. This is a small thing, but it helps usability because anyone accessing the page can immediately see that this section is for managing the services directory. After that, the page is split into three separate sections: one for adding a gym, one for adding a coach

and one for adding a professional. I chose to separate them clearly because each type of service needs different fields, and putting everything into one large form would have made it more confusing to use. Each section uses its own HTML form, and all of the forms use the POST method. This is important because the page is sending data to the server so that it can be processed and stored in the database. Using POST is the correct choice here because these are changes being made to the system, not just data being viewed. It also avoids putting submitted values directly into the URL. A hidden input field with the name "form_type" is included in each form. This is one of the most important sections of the webpage, as it allows the backend to know what type of form is being submitted. For example, one form submits a "gym" type, another submits "coach," and another submits "professional." I did this so that one route can handle multiple forms instead of creating a new route for each type of service. This keeps the code organized and avoids unnecessary duplication.

The gym form has fields such as gym name, location, and facilities. The name field is marked as required since every gym needs to have at least an identifiable name. However, location and facilities are marked as optional, which provides more flexibility in case not all information is readily available. For the coach form, it has a similar format to the gym form but uses more appropriate field names such as coach name, specialisms, and location. In a similar manner, the professional form has fields such as name, profession, and location to allow other support services to be included in the system as well.

I used placeholder text inside the inputs to make it clearer what the user is expected to enter. This improves the usability of the page and reduces the chance of confusion or badly formatted entries. I also kept the layout simple, using headings and horizontal lines to separate each form. That makes the page easier to scan and helps stop the different service sections from blending together. Overall, this page is an important part of the system because it provides a clear interface for entering service data before it is processed by the backend. It supports the wider services directory feature of BrawlBase and shows how the front end was designed to work directly with the backend route and database structure. By using multiple forms, hidden form identifiers and organised inputs, this page helps make the system easier to manage and extend later.

```
37
38 <hr>
39
40 <form method="post">
41   <input type="hidden" name="form_type" value="seed">
42   <button type="submit">Add Example Data</button>
43 </form>
44
45 {% endblock %}
46
```

I have added another form at the bottom of the admin services page. This form is used to insert example data into the system. This form is much simpler compared to the previous ones. This is because this form contains only a hidden input field and a submit button. The aim of this feature is to add example data into the system. This makes it much simpler to test

and demonstrate the website's functionality without having to manually add multiple gyms, coaches, or professionals.

The form uses a POST method, just like all the other forms in this page, because it is sending data to be processed in the backend. There is a hidden input element in this form with a name attribute equal to `form_type` and a value equal to `seed`. This is working in a similar fashion to the other forms in this application. In this case, when `form_type` equals `seed` in this form, it will be understood that a set of predefined example records will be inserted into the database. The button with the text "Add Example Data" will trigger this action when clicked. This will enable the database to be populated instantly with a number of records, which will be helpful in development and testing phases. For example, instead of manually inserting a number of gyms and a number of coaches into the database in order to test whether search and display functions are working as desired, this will be done automatically by this function.

This feature is also useful in demonstrating the project, as there is always data available for demonstration of the complete functionality of the project. The website could be empty, indicating that there are no data entries yet, without the inclusion of this feature. The inclusion of the seeding function makes the project easier to test, more efficient in use, and reliable in the demonstration of the final project. Inclusion of the seeding function makes the project more useful, as there is consideration for testing the project, as the process of repeatedly inputting data while in development is reduced.

Search Page (search.html) – Partner Filtering System

```

templates > <> search.html > ...
1  {% extends "base.html" %}
2  {% block content %}
3      <h2>Search Partners</h2>
4
5      <form class="form" method="get">
6          <label>Martial Art</label>
7          <input name="martial_art" value="{{ martial_art }}">
8
9          <label>Skill Level</label>
10         <input name="skill_level" value="{{ skill_level }}">
11
12         <label>Location</label>
13         <input name="location" value="{{ location }}">
14
15         <button type="submit">Search</button>
16     </form>
17
18     <h3>Results</h3>
19     <div class="list">
20         {% for r in results %}
21             <div class="row">
22                 <div>
23                     <strong>{{ r.username }}</strong>
24                     <div class="muted">{{ r.martial_art }} | {{ r.skill_level }} | {{ r.location }}
25                 </div>
26                 <div>
27                     <a href="/profile/{{ r.username }}">View</a>
28                 {% if user and user.username != r.username %}
29                     <a href="/messages?to={{ r.username }}">Connect</a>
30                 {% endif %}
31             </div>
32         </div>
33         {% else %}
34             <p class="muted">No results.</p>
35         {% endfor %}
36     </div>
37 {% endblock %}

```

This file is used for the search function in BrawlBase, allowing users to find new partners for training based on certain criteria such as the martial art, skill level, and location. It is one of the main functions of the program, as it directly relates to the main function of the program, which is connecting users with like-minded individuals based on certain criteria. It extends the base.html file, which makes sure that the page looks the same as the rest of the site, eliminating the need for repetition in the navigation, design, etc. The content of the page is inside the {% block content %}, which makes sure the page fits in with the rest of the structure of the program. At the top of the page, a form is provided that allows users to input their search criteria. However, this form uses a GET request instead of a POST request. This is because this form does not alter any data in the system; it simply retrieves and displays existing data in the system. It would be more appropriate to use a POST request in a search form in a real application because it does not display search parameters in the URL. The form provided in this application contains three input fields: martial art, skill level, and location. Each input field is assigned a name attribute that corresponds to what is required

by the route provided in this application. This allows users to access the values input in this form by using request arguments in a route in a Flask application. The value attributes are populated using Jinja syntax such as `{{ martial_art }}`, which ensures that when a search is made in this application, previously input values in this form are displayed in the input fields. Once this form is submitted, it is processed in the backend, which then returns a list of users that match this input. This is then passed to the template as a variable named `results`. This page then uses a Jinja templating engine's `for` loop to cycle through each result and display the necessary information to the user. For each user, their username is displayed along with their martial art, skill level, and location. This gives the user enough information to determine whether this match is appropriate without cluttering the screen with too much information. Each result also includes a link to their profile page, as well as an option to connect to this person through the messaging system if this result is not their own profile. This is achieved through a logical validation statement: `{% if user and user.username != r.username %}`. This adds a small but important logical validation to the interface, which improves the overall user experience.

If there are no users, the page simply shows "No results." This is important so that the user doesn't think something went wrong, as they would with an empty page. From the programmer's point of view, this page shows the use of dynamic data from the backend, which can be used in real-time with the help of Jinja templating. It also shows the use of user input in the determination of the data that is sent from the backend. This is slightly more advanced compared to the static page, as the data changes based on the input provided in the search query. From the design point of view, this is efficient as there is no need for extra data processing on the frontend side. The backend does all the data filtering, so the data sent to the frontend is the relevant data, which makes the process faster, especially as the data in the database increases in size. The structure of the form is also very easy to expand in the future, for example, in the case of the addition of extra filters like the level of experience or the type of training goals.

Overall, this feature is an important part of the functionality of BrawlBase. This is because it utilizes the input given by the user and the processing and rendering to create a feature that is useful and efficient. This feature helps the user find a training partner, thus aiding in the overall purpose of the project and showing a higher level of technical implementation.

Search Page (search.html) – Backend Filtering System

```

98     @app.route("/search", methods=["GET", "POST"])
99     def search():
100         martial_art = request.values.get("martial_art", "").strip()
101         skill_level = request.values.get("skill_level", "").strip()
102         location = request.values.get("location", "").strip()
103
104         q = User.query
105         if martial_art:
106             q = q.filter(User.martial_art.ilike(f"%{martial_art}%"))
107         if skill_level:
108             q = q.filter(User.skill_level.ilike(f"%{skill_level}%"))
109         if location:
110             q = q.filter(User.location.ilike(f"%{location}%"))
111
112         results = q.order_by(User.username.asc()).all()
113         return render_template(
114             "search.html",
115             user=current_user(),
116             results=results,
117             martial_art=martial_art,
118             skill_level=skill_level,
119             location=location
120         )

```

This piece of code is used to handle the backend of the search feature of BrawlBase. While the search.html file is used to provide a platform where users can type their search parameters, this route is used to process those parameters and retrieve the appropriate data from the database. This is one of the main features of this system, as it enables users to find training partners that fit their requirements, thus helping them achieve their goal.

This route is defined using: `@app.route("/search", methods=["GET", "POST"])`

This enables this function to handle both showing this page to the user and handling the search process. In this case, the search is done using GET requests, as data is being retrieved rather than created. However, including both is useful, especially if one needs to expand this feature later.

The first part of this function retrieves the values entered by the user using `request.values.get()`. This is done irrespective of whether the form was submitted using GET or POST, giving us access to this data. Each of these is then followed by `.strip()`, which removes any extra spaces that may have been entered by the user at the beginning or end of their data. This is a small, but important, step, as it prevents any problems that could arise if the user accidentally enters spaces while trying to search. A base query is then established: `q = User.query` This is the original data set, which includes all users. From here, various filters are applied depending on whether the user has entered any data into certain fields or not. This prevents any unnecessary filtering, keeping the query efficient. The filters

are implemented by using if statements. If a martial art has been filled in by the user, then this filter is added to the query through: `User.martial_art.ilike(f"%{martial_art}%")`

By using `ilike` instead of `like`, it is case-insensitive. This is important because it does not require users to enter exact case when searching for a user. The `%` symbols are wildcards that allow partial matches to be made. This allows users to search for partial strings, such as “box” instead of “boxing.” It also allows multiple filters to be implemented by searching by skill level and location.

This is an efficient way of constructing the query since it will only apply the filters that it needs. At the same time, it is easy to maintain the code since any additional filters can be easily added by including additional conditions in the query. Once all the filters have been applied to the results, they need to be sorted. This is done by the line of code: `results = results.order_by(User.username.asc()).all()`. It is important to sort the results so that they can be easily displayed. This is because it will make the user experience better since the results will be easy to read. The final part of the function is to send the results back to the template. This is done with the line of code: `return render_template("index.html", results=results, search=search, location=location, age_group=age_group)`

In addition to sending the results, the search parameters are also sent back. This is so that the input fields in the frontend will have the previous search parameters of the user. This will make it easy for the user to refine their search without having to start all over. From a technical point of view, this piece of code is also significant in that it demonstrates a number of fundamental ideas related to building queries dynamically using SQLAlchemy. This is significant in that it prevents any form of redundant data from being sent to the client-side, which is essential if scalability is to be achieved, especially if there is an increase in the number of clients.

Conclusively, this piece of code is essential in that it ensures that the application is user-oriented and functional. This is because, through this piece of code, there is a connection between the users and the query, which is essential in ensuring that only relevant output is sent to the client. This is efficient, flexible, and thus a good example of how the backend can be utilized to ensure that a critical feature is supported.

Messaging Route (/messages) – Backend Communication System

```

175
176 @app.route("/messages", methods=["GET", "POST"])
177 def messages():
178     if not login_required():
179         return redirect(url_for("login"))
180
181     user = current_user()
182     prefill_receiver = request.args.get("to", "")
183     if request.method == "POST":
184         receiver = request.form.get("receiver", "").strip()
185         content = request.form.get("content", "").strip()
186         if not receiver or not content:
187             flash("Receiver and message are required.")
188         else:
189             msg = Message(
190                 sender_username=user.username,
191                 receiver_username=receiver,
192                 content=content
193             )
194             db.session.add(msg)
195             db.session.commit()
196             flash("Message sent.")
197
198     inbox = Message.query.filter_by(receiver_username=user.username).order_by(Message.timestamp.desc()).all()
199     sent = Message.query.filter_by(sender_username=user.username).order_by(Message.timestamp.desc()).all()
200     return render_template(
201         "messages.html",
202         user=user,
203         inbox=inbox,
204         sent=sent,
205         prefill_receiver=prefill_receiver
206     )
207
208     return app

```

This part of code is responsible for managing the messaging feature in BrawlBase. This part of code is important in this application as it allows users to communicate directly with each other, which aligns with the main goal of this application to help users find training partners.

The code is defined as `@app.route("/messages", methods=["GET", "POST"])`

This allows this function to handle both displaying a message and sending a new message. When this page is visited, it is a GET request, which means it displays the user's inbox as well as their sent messages. When a new message is sent, it is a POST request, which means it is processed as a new message.

To start with, there is a check done using the function `login_required()`. This ensures that only those users who have logged in can use the messaging system. If a user is not logged in, they are redirected to the login page. The current user is then accessed using the function `current_user()`. This is essential, as the system needs to know who is sending messages. Another variable, `prefill_receiver`, is accessed using the function `request.args.get("to")`. This is essential, as the system needs to automatically fill in the receiver's name when the user clicks the "Connect" button, which is accessed from the search page. When the method is POST, data is processed. The receiver's name and message content are accessed using the function `request.form.get()`. The `strip()` function is then applied to remove any spaces. A validation is then done to ensure that the receiver's name and message content have been filled in. If not, an error message is displayed using

`flash()`. If the validation is successful, then a new `Message` object is created. Here, the `sender_username` is set to the current user's username, while the `receiver_username` is set to the username entered by the user. Finally, the content of the message is saved. This is an object that represents a message, which is then saved to the database.

This message is then saved to the database using `db.session.add()` to add the message to the database, while `db.session.commit()` is used to commit the changes to the database, thus ensuring that the message is not lost. Finally, a message is displayed to the user indicating that the message was successfully sent. Messages are then retrieved to be displayed. To do this, messages in the inbox are queried. This is done by using `query.filter_by(sender_username=current_user.username)` to retrieve messages where the receiver's username is the current user's username. On the other hand, messages that have been sent by the user are retrieved using `query.filter_by(receiver_username=current_user.username)`. Both messages are sorted using `order_by(Message.timestamp.desc())` so that messages are displayed with the latest messages appearing first. This is done to ensure that messages displayed to the user are those that are most relevant.

Lastly, the data is sent to the `messages.html` page using `render_template()`. This data sent is the current user, inbox messages, sent messages, and the receiver value. This enables the frontend to dynamically use this data to create a complete user interface for the user. Technically speaking, this route encompasses a number of key concepts in handling forms, validation, database queries, filtering, among others. This route also encompasses the handling of multiple database queries in retrieving data in different forms or views. This is exemplified in the inbox messages and sent messages. This is achieved without any repetition in the messages sent, as a message sent can be sent and received at the same time.

Handling both the sending and receiving of messages in this single route ensures that the system is simple and organized. Moreover, the use of database filtering is effective in ensuring that the system is efficient in performance. This is because the system does not retrieve any message in the database but only relevant messages. This is effective in ensuring that the system scales well for a large number of users.

This section of code, in general, is a critical piece of code that facilitates user interaction within the system. This is because it connects user interaction with data storage and display, creating a "communication system" if you will. This is a much more advanced piece of functionality, greatly increasing the utility of BrawlBase.

Base Template (base.html) – Shared Layout and Navigation System

```

1  <!doctype html>
2  <html lang="en">
3  <head>
4      <meta charset="utf-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1" />
6      <title>BrawlBase</title>
7      <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
8  </head>
9  <body>
10     <header class="nav">
11         <div class="brand"><a href="/">BrawlBase</a></div>
12         <nav class="links">
13             <a href="/search">Search</a>
14             <a href="/services">Services</a>
15             {% if user %}
16                 <a href="/dashboard">Dashboard</a>
17                 <a href="/messages">Messages</a>
18                 <a href="/profile/{{ user.username }}">My Profile</a>
19                 <a href="/logout">Logout</a>
20             {% else %}
21                 <a href="/login">Login</a>
22                 <a href="/register">Register</a>
23             {% endif %}
24         </nav>
25     </header>
26
27     <main class="container">
28         {% with msgs = get_flashed_messages() %}
29             {% if msgs %}
30                 <div class="flash">
31                     {% for m in msgs %}<div class="flash-item">{{ m }}</div>{% endfor %}
32                 </div>
33             {% endif %}
34         {% endwith %}
35
36         {% block content %}{% endblock %}
37     </main>
38
39     <footer class="footer">
40         <small>BrawlBase – Martial Arts Search Platform</small>
41     </footer>
42 </body>
43 </html>

```

This file is essentially the main layout template for the entire BrawlBase application. Instead of having the same HTML code for all the pages of the application, this template is extended by all the other pages. This is more efficient and also allows for consistency throughout the system.

The code starts with the basic structure of HTML. This includes the doctype declaration, the language setting, and the meta tags for the character encoding and responsive design. The stylesheet is linked using the `url_for` function provided by the Flask framework. This function automatically generates the appropriate path for the static file. This is more reliable than hardcoding the paths and also allows for easier extensibility of the application. The navigation bar is defined within the header. This navigation bar has links for the search and services pages. There is also a conditional statement that checks whether the user is logged

in or not. If the user is logged in, they are provided with more options such as the dashboard, messages, profile, and logout. If no user is logged in, the login and register options are provided. This conditional rendering improves usability as well as security. For instance, a user who has not logged in cannot access messaging or a dashboard through the navigation bar. The main content area is defined through a block named content. This allows other templates to include their own content while at the same time reusing the same layout as this template. This is an example of template inheritance in a Flask application. This minimizes code duplication, making it easier to maintain this system. If a change is to be made to this layout, it can be achieved in just one place as opposed to several places if it were duplicated.

Apart from this, flash messages are also being processed in this template. In this case, it is important to note that the system will be able to get the flashed messages and then display them to the user. This is another significant aspect that will be helpful in sending messages or filling out forms. It is included in this general template in order to ensure that a consistent user interface is provided throughout the application.

Finally, it is important to include the footer at the end of the application. This is a simple aspect but will be helpful in ensuring that the identity of the application is clear. From a technical standpoint, this template is significant in that it is a clear example of a comprehensive application that will be helpful in developing a comprehensive frontend system. This is because aspects such as navigation and messages are centralized in this application template. Overall, this file plays a very important role in ensuring that there is a certain level of consistency throughout the application. This is due to the fact that it ensures that every page has a certain look while at the same time allowing for flexibility through content blocks. This makes it a more organized application.

Home Page Template (index.html) – Main Entry Point of the System

```

1  {% extends "base.html" %}
2  {% block content %}
3      <h1>Martial Arts Search Platform</h1>
4      <p>Connect with sparring partners, gyms, coaches, and sports professionals.</p>
5      <div class="grid">
6          <a class="card" href="/search">Find Partners</a>
7          <a class="card" href="/services">Find Gyms, Coaches, Professionals</a>
8          {% if user %}
9              <a class="card" href="/dashboard">Go to Dashboard</a>
10             {% else %}
11                 <a class="card" href="/register">Create Account</a>
12             {% endif %}
13     </div>
14 {% endblock %}
15

```

The file is used as the home page for the BrawlBase application. The home page is the first page that the user sees when they visit the website. It also serves as an introduction for the user regarding the purpose of the application. The primary use of this file is to direct the user towards the most important areas of the application. For instance, the user may search for training partners or use the services provided by the application and the user's dashboard. At the top of the file, the page extends base.html. This means that the page uses the same layout and navigation as the other pages of the application. I have used this method because the structure of the website will be the same for all the pages. I do not have to repeat the same HTML code multiple times. This way, I do not have to make the same changes for all the pages.

Inside the {% block content %} section, I have added a main heading and a paragraph of text that describes what the platform is for. This heading immediately lets the user know what the website is for, as it is a search platform for martial arts. The paragraph of text gives a quick overview of what types of connections the user can make using the system, whether it is for sparring partners, gyms, coaches, or other sports professionals. This helps the user understand the purpose of the website from the moment the page loads. The main content of the webpage is inside a <div class="grid">, which is a type of structure for organizing the links. These are the main content items of the webpage, as they are displayed as cards. These cards allow the user to navigate the webpage, whether it is to search for a training partner or view the gyms, coaches, and professionals.

A conditional Jinja statement is also used here. If the user is logged in, they are provided with a card that links them directly to the dashboard. On the other hand, if the user is not logged in, they are provided with a card that links them to the register page. This has made the page dynamic. The dynamic nature of the page has improved its usability. This is because the options provided are based on the current state of the user. The logged-in user is directed to the personalised part of the platform. On the other hand, the visitor is prompted to register and use the services provided by the platform. The use of this code is important because it prevents the page from displaying unnecessary information. This code also assists the larger system with the authentication of the users. For instance, the system allows the users who are already registered and logged in to the platform to reach the

personalised part of the platform easily. On the other hand, the system allows the first-time visitors to the platform to easily register and use the services provided by the platform.

From a technical point of view, it is a good example of the use of template inheritance, conditional rendering, and structured navigation. Although it is a simple page, it is an important one as it serves as the main entrance of the application. It helps tie the main features of the platform together, hence providing a better user experience for the users of the application. Overall, it is a good example of a webpage as it introduces the user to the platform and gives access to the main features of the application. It is also useful for both old and new users of the application as it helps make the structure of the system easy to understand from the beginning.

Profile Page (profile.html) – User Information Display and Interaction

```

1  {% extends "base.html" %}
2  {% block content %}
3      <h2>Profile: {{ profile.username }}</h2>
4      <p><strong>Martial art:</strong> {{ profile.martial_art }}</p>
5      <p><strong>Skill level:</strong> {{ profile.skill_level }}</p>
6      <p><strong>Location:</strong> {{ profile.location }}</p>
7
8      {% if user and user.username != profile.username %}
9          <p><a href="/messages">Message this user</a></p>
10     {% endif %}
11 {% endblock %}
12

```

This file is in charge of displaying a user's individual profile in the BrawlBase application. This allows users to view important information about other users, such as their martial art, skill level, and location. This is an important aspect of the application, as it helps a user determine if another user is a good training partner before making any contact with them. The template is based on base.html, meaning it follows the same structure as the other pages in the application. This is an important aspect of web development, as it prevents repetition of certain items, such as the navigation bar, across multiple pages. This content is placed inside a `{% block content %}` area.

The username of the user is shown at the top of the page as a heading. This indicates clearly who the profile belongs to. At the end of this heading, some important characteristics of the user are shown, such as their martial art, skill level, and location. This is shown through Jinja templating, as shown in `{{ profile.martial_art }}` and `{{ profile.location }}`. A conditional statement has been used to show whether a user can message this person or not. This is determined by whether a user is logged in and whether this profile belongs to someone else. This prevents a person from trying to message themselves, which would make no sense in this context.

This conditional logic helps improve usability as well as system integrity. It ensures that only proper actions are available for the user, and it removes any unnecessary or invalid options from the user interface. This also directly relates to the messaging system, as it allows the user to go from looking at a profile to communicating with someone in a simple and intuitive

manner. From a technical standpoint, this webpage shows the rendering of dynamic data, as well as conditional logic within a template. It also shows the connectivity of different areas of the system, as it directly relates the search system (where users are found) with the messaging system (where users communicate with one another). From a design standpoint, the webpage is simple and only includes relevant information, as it is only displaying the most relevant information needed for a user's decision-making.

In summary, this feature makes the platform more interactive as the user is able to explore other profiles and take action based on the information they have. This feature makes the platform more connected and user-centered as it achieves the objective of helping users communicate with appropriate training partners.

Services Page (services.html) – Displaying Gyms, Coaches and Professionals

```

templates > <> services.html > ...
1  {% extends "base.html" %}
2  {% block content %}
3      <h2>Services Directory</h2>
4
5      <h3>Gyms</h3>
6      <div class="list">
7          {% for g in gyms %}
8              <div class="row"><div><strong>{{ g.name }}</strong> <span class="muted">{{ g.location }}</span></div></div>
9          {% else %}
10             <p class="muted">No gyms added yet.</p>
11          {% endfor %}
12      </div>
13
14      <h3>Coaches</h3>
15      <div class="list">
16          {% for c in coaches %}
17              <div class="row"><div><strong>{{ c.name }}</strong> <span class="muted">{{ c.specialism }} | {{ c.location }}</span></div></div>
18          {% else %}
19              <p class="muted">No coaches added yet.</p>
20          {% endfor %}
21      </div>
22
23      <h3>Professionals</h3>
24      <div class="list">
25          {% for p in pros %}
26              <div class="row"><div><strong>{{ p.name }}</strong> <span class="muted">{{ p.profession }} | {{ p.location }}</span></div></div>
27          {% else %}
28              <p class="muted">No professionals added yet.</p>
29          {% endfor %}
30      </div>
31  {% endblock %}
32

```

This file is used for rendering the services directory page in the BrawlBase system. This allows users to view various types of martial arts-related services. This feature is in line with the main objective of the system as users are given an opportunity to train in gyms as well as seek guidance from professionals. It extends the base.html file. This means that the layout is consistent with the entire application. This is important as it ensures that certain elements such as the navigation bar are not repeated. The page is divided into three sections. These include gyms, coaches, and professionals. All these sections have used the Jinja for loop function. This function is used for looping through the data received from the backend. For example, in the gyms section, the loop is used to display the name as well as

the location of the gym. Similarly, in the coaches section, the loop is used to display the specialism as well as the location. Similarly, in the professionals section, the loop is used to display the profession type as well as the location.

The `{% else %}` statement is also included in the loop. This is done to show a message when there is a lack of data for a specific category. For example, “No gyms added yet” will be displayed. This is useful from a user interface perspective, as it is better than a blank screen, which may confuse the user. From a technical perspective, it can be seen that this page is displaying multiple datasets. This is done by using a single template. The data is displayed in different categories, which is useful for the user. This is because it is more easily understood. It can also be seen that this page is connected with the admin services. This is because the data displayed on the webpage is entered through the admin. This means that the webpage is displaying real-time data. This is useful from a technical perspective, as it can be seen that different features of a website can interact with each other. For example, one feature is for adding data, and another is for displaying it.

This feature contributes to the completeness of this system by expanding its functionality beyond user-to-user interaction. It allows users to explore professional services within martial arts, making it more useful and realistic. The structured format and content will allow it to be extended in the future, for example, by incorporating filters and ratings for each service provided.

Configuration File (config.py) – Application Setup and Security

```

config.py > ...
1  import os
2
3  BASE_DIR = os.path.abspath(os.path.dirname(__file__))
4
5  class Config:
6      SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-change-me")
7      SQLALCHEMY_DATABASE_URI = os.environ.get(
8          "DATABASE_URL",
9          "sqlite:/// " + os.path.join(BASE_DIR, "brawlbase.db")
10     )
11     SQLALCHEMY_TRACK_MODIFICATIONS = False
12

```

This file is used for the configuration of important settings for the application. Instead of hard-coding the values in the application, I have used this file for the same purpose. This has helped in the better organisation of the code. The variable `BASE_DIR` is used for finding the absolute path for the project directory. This has helped in the creation of the database file in the same directory irrespective of the location of the application. This has made the system more portable. A configuration class has been defined for the important settings of the application. The `SECRET_KEY` is used by the Flask application for the purpose of security. It is used for the protection of important data such as login information. The environment variable `os.environ.get()` has been used for the same purpose. The default

value has been provided for the same. This has helped in the improvement of the security of the application. The database connection has been defined by the `SQLALCHEMY_DATABASE_URI`. SQLite has been used for the same. The database file has been created locally. The path for the same has been provided by the `os.path.join()` function. `SQLALCHEMY_TRACK_MODIFICATIONS` is set to `false`, which prevents any unnecessary tracking features. This is done to enhance performance, as there is no need to track any changes made to the data.

From a technical standpoint, this file shows good software design, where configuration is kept separately. This is helpful, as it becomes easier to manage, test, and expand the system. For example, if you want to change your database or secret key, you don't have to edit any of your main code. This is a well-structured application, which is good for security, especially for something like BrawlBase.

BrawlBase – Testing

Login Testing

The login functionality was tested to ensure that only registered users with valid credentials could access the restricted parts of the BrawlBase system. This was one of the most important features to test because the dashboard, messaging system and other personalised features depend on successful authentication. Both valid and invalid inputs were used so that the feature could be tested under normal conditions, boundary conditions and erroneous conditions.

Black Box Testing – Login

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
Program initially displays the login page	Open the application and navigate to <code>/login</code>	The login page should load correctly with visible username and password fields	The login page loaded correctly and both fields were visible	PASS
User can navigate to the login page from the sign-up page	Open the register page and click the login link	The system should redirect the user to the login page	The user was redirected correctly to the login page	PASS
User can enter login details into the form	Enter a username and password into the input fields	The form should accept the entered data without any issue	Username and password could both be entered correctly	PASS
Login works with valid details	Username: Hamza1, Password: Secure123	The system should log the user in and redirect to the dashboard	The user was redirected to the dashboard successfully	PASS
Access is denied if the password is incorrect	Username: Hamza1, Password: WrongPass	The system should not allow login and should display an error message	Login was denied and an error message was shown	PASS

Access is denied if the username does not exist	Username: FakeUser, Password: Secure123	The system should not allow login and should display an error message	Login was denied and an error message was shown	PASS
Empty login form should not be accepted	Leave username and password blank and click login	The form should not submit because both inputs are required	The browser prevented submission due to required fields	PASS
User can retry after failed login	Enter incorrect password, then re-enter correct password	The user should be able to retry immediately and log in successfully	The system allowed a retry and login then worked correctly	PASS
Restricted page cannot be accessed without login	Type <code>/dashboard</code> directly into the URL without logging in	The system should redirect the user back to the login page	The user was redirected to the login page	PASS
Login session remains active after successful login	Log in successfully, then navigate between dashboard and messages	The system should remember the logged-in user	The user stayed logged in across pages	PASS
Very long username input	Username: 100+ characters	System should handle input without crashing and reject invalid login	System handled input and rejected login correctly	PASS
Special characters in username	Username: <code>@!#\$\$%</code>	System should not crash and should reject login	System rejected login correctly	PASS
SQL injection attempt	Username: <code>' OR '1'='1</code> , Password: anything	System should not allow authentication bypass	Login failed and no access was granted	PASS

Leading and trailing spaces in username	Username: "Hamza1 "	System should trim spaces and process login correctly	Login worked correctly after trimming spaces	PASS
Multiple rapid login attempts	Enter incorrect password repeatedly (5–10 times)	System should remain stable and allow retries	System remained stable and allowed repeated attempts	PASS

Black Box Testing Commentary – Login

The test for the login system involved the usage of different normal and invalid data in the input fields. This ensured that the login system functioned as expected in all cases. The test began with the login page. This involved opening the login page to ensure that it displayed correctly and had the username and password input fields. This test passed, indicating that the login page is accessible and functional at the start of the application.

The next test involved a correct login. This involved the usage of the correct username and password. The expected result of this test was that the system should authenticate the user and redirect them to the dashboard. This test passed, indicating that the login route correctly checks the entered details and redirects the user to the dashboard if the entered details match the details in the database. The system was also tested with incorrect credentials. One test used an existing username but with the wrong password. Another test used a username that did not exist in the database. In all cases, the user was denied login and an error message was displayed. This proved that the system does not allow unauthorized users to enter the restricted areas of the system. Another significant test was performed by leaving the form fields empty. Since the form fields are marked as required in the HTML code, the form was not submitted. The browser canceled the request before it reached the server. This enhanced the usability of the system because the user was immediately notified that the form was not fully filled out. The ability to retry a login after failure was also tested. This is important since sometimes a user might make a simple mistake in entering their password. This system allowed retries without crashing or freezing, which indicated that it remained stable even after a mistake.

Lastly, the dashboard page was accessed directly through the URL without being logged in. This test was important since it ensured that the system protected access to certain pages as required. This test resulted in the user being redirected to the login page, which indicated that this system worked as expected. Black box testing indicated that this system worked as expected, whether the conditions were valid or not.

White Box Testing – Login

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS / FAIL
The login route accepts user input	<code>request.form.get("username"), request.form.get("password")</code>	Confirms that the system retrieves user-entered data from the form	PASS
Username input is cleaned before processing	<code>.strip()</code>	Removes accidental spaces and prevents incorrect login failures	PASS
The system queries the database for the user	<code>User.query.filter_by(username=username).first()</code>	Confirms that the system checks whether the user exists in the database	PASS
The system validates password securely	<code>user.check_password(password)</code>	Ensures that hashed passwords are used instead of plain text comparison	PASS
Login only succeeds if both user exists and password matches	<code>if user and user.check_password(password):</code>	Prevents access when either condition is not satisfied	PASS

Session is created on successful login	<code>session["username"] = user.username</code>	Confirms that the system stores user session data for persistence	PASS
User is redirected after successful login	<code>return redirect(url_for("dashboard"))</code>	Ensures correct navigation to protected dashboard after login	PASS
Error message is displayed on failed login	<code>flash("Invalid username or password.")</code>	Confirms that the system provides feedback when authentication fails	PASS

White Box Testing Commentary – Login

White box testing was carried out to ascertain if the internal logic of the code of the login page had been correctly programmed. This involved testing the route as well as the code conditions as opposed to just testing the page's behavior. The first important part of the code in the login page is where it retrieves the form data. This is clear from the use of `request.form.get()` to obtain the username as well as password entered into the system by a user. It can also be seen that there is a use of `.strip()` to remove extra spaces at the beginning as well as end of the entered username, which can help in preventing failure to log in due to spacing.

The next significant part of the code is the query sent to the database. This is shown by the line using `User.query.filter_by(username=username).first()`. This shows that the system is trying to find the user in the database using the username provided in the form. This is significant because, without finding the user, the password cannot be verified. The condition `if user and user.check_password(password):` is probably one of the strongest parts of the login code. This shows that, without finding the user in the database, the system would not allow the user to log in, and that, even if found, the password provided by the user would have to match the password found in the user's account. Finally, the code saves the username in the session, showing that, having found that the user is valid, the system is able to remember the user even after they have logged in, thus enabling them to move from one page to another in the system. The redirect is then shown, indicating that, having found that the user is valid, the system is able to redirect the user, thus showing that the user has successfully logged in. The last part of the route handles failures. If the username is not found or if the password is incorrect, a flash message is displayed, and then the user is

redirected back to the login page. This is good from a security standpoint, as well as a usability standpoint, because you're not giving any information about what specifically happened. Overall, white box testing shows that the internal logic of the login system is correct. This feature is coded to retrieve data, safely validate that data against the database, create a session, or reject invalid attempts.

It is clear that all the tests performed on the login function were a success. When correct details were given, the system performed as required. When incorrect details were given, the system performed as required by denying unauthorized access. Also, denying unauthorized access to certain pages by redirecting those users to the login page is a success. This means that black box testing and white box testing were performed, which means the system is secure, reliable, and appropriate for use in the project.

Register Testing

The registration functionality was also tested, with the aim of ensuring that a new user can successfully create an account and gain access. This is an important functionality of the system, as it relies on the user being registered in the database for other functions, such as logging in, accessing the dashboard, and communicating, to be available. For the functionality of the system, it is important that it is tested with both correct and incorrect data, allowing it to be tested under normal, boundary, and error cases.

Black Box Testing – Register

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
Program provides access to the sign-up page	Open the application and navigate to <i>/register</i>	The registration page should load correctly with all input fields visible	The registration page loaded correctly and all input fields were visible	PASS
User can navigate to the sign-up page from the login page	Open the login page and click the register link	The system should redirect the user to the register page	The user was redirected correctly to the register page	PASS

User can enter registration details	Enter a username, password, martial art, skill level and location into the form	The form should accept the data entered by the user	All fields accepted input correctly	PASS
Valid registration with all fields completed	Username: Hamza1, Password: Secure123, Martial Art: Boxing, Skill Level: Beginner, Location: Slough	A new user account should be created and saved in the database	The account was created successfully and saved in the database	PASS
Valid registration with only required fields completed	Username: NewUser1, Password: Strong123, leave optional fields blank	Account should still be created successfully	Account created successfully with optional fields left blank	PASS
Duplicate username is rejected	Username: Hamza1, Password: Another123	System should reject registration and display an error message	Registration was rejected and an error message was shown	PASS
Password shorter than 6 characters is rejected	Username: ShortPassUser, Password: 12345	System should reject password and display validation message	Registration was rejected and an error message was shown	PASS
Username shorter than 3 characters is rejected	Username: ab, Password: Secure123	System should reject username and display validation message	Registration was rejected and an error message was shown	PASS
Empty required fields are not accepted	Leave username and password blank and try to submit	The form should not submit because required fields are missing	The browser blocked submission because required fields were empty	PASS

Username with leading/trailing spaces	Username: " Hamza2", Password: Secure123	System should trim unnecessary spaces and process the registration correctly	Username was trimmed and account created correctly	PASS
Very long username input	Username: 100+ characters, Password: Secure123	System should not crash and should reject or handle input safely	System remained stable and handled input correctly	PASS
Special characters in username	Username: @!#\$%\$, Password: Secure123	System should not crash and should handle the input safely	System remained stable and registration was handled safely	PASS
SQL injection style input	Username: ' OR '1'='1, Password: Secure123	System should not allow any bypass or abnormal behaviour	Registration did not bypass validation and system remained stable	PASS
Password field hides typed characters	Enter a password into the password input	Characters should not be visible on screen	Password was hidden correctly	PASS
Multiple registration attempts after failure	First use invalid data, then retry with valid data	System should allow retry and then create account successfully	Retry worked correctly and valid account was created	PASS

Black Box Testing Commentary – Register

The system has been tested under a variety of valid and invalid input values to ascertain whether it performs as expected in every case. To start off, the system has been tested by opening up the registration page to see whether it loads correctly and includes all necessary input fields such as a username and password. This test has been a success, which means that the registration page loads correctly to start off the account creation process. The next test involved registering a new user with valid input values such as a username, password,

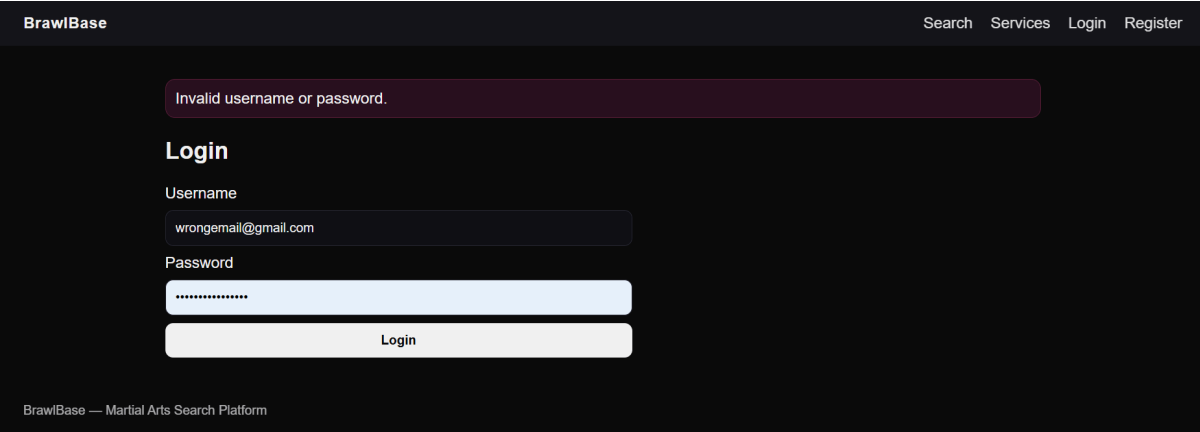
as well as other optional input values such as martial art, skill level, and location. This test has also been a success, which means that the registration route handles valid input values correctly to insert a new record into the database table containing user records.

The system was then subjected to tests with invalid data. In one of the tests, an attempt was made to register a new account with a username that had already existed in the system. In this case, the system rejected the attempt and displayed an error message. This proved that the system does not allow duplicate accounts and that each username is unique. In the second test, an attempt was made to register with a password that was too short. The system rejected the input and displayed a validation message. This proved that the system does not allow weak passwords. Further tests were carried out with invalid usernames. In one of the tests, an attempt was made to register with a username that was too short. The system rejected the input, thus proving that the system had correctly implemented the validation rules. In the next test, an attempt was made to leave the required fields blank. The fact that the fields were marked as required in the HTML meant that the browser did not allow the form to be sent. This improved the usability of the system since the user was immediately informed that the form had not been filled in.

The system was also tested with unusual input values such as usernames containing special characters and long usernames. This testing was important to ensure that the system does not crash when unusual input values are entered into the system. Lastly, multiple registration tests were carried out by first entering wrong information into the system and then attempting to register again with correct information. This test proved that the system allows a user to retry registering into the system without any problems, as it successfully registered the user on the second attempt.

From this case study, it is clear that black box testing proved that the system can function as expected under both valid and invalid input values. This is because it can create new accounts, prevent duplication of usernames, as well as validate usernames, making it a reliable system to use in a real application.

Wrong Password/Username



The screenshot displays the BrawlBase login interface. At the top, the site name 'BrawlBase' is on the left, and navigation links 'Search', 'Services', 'Login', and 'Register' are on the right. A prominent red error message box at the top center reads 'Invalid username or password.' Below this, the 'Login' section is titled. It contains two input fields: 'Username' with the text 'wrongemail@gmail.com' and 'Password' with masked characters. A 'Login' button is positioned below the password field. At the bottom left, a footer reads 'BrawlBase — Martial Arts Search Platform'.

Duplicate Username

BrawlBase

Search Services Login Register

Username already exists.

Register

Username

Password (min 6)

Martial Art

Boxing, MMA, Kickboxing

Skill Level

Beginner, Intermediate, Advanced

Location

City or postcode

Create account

Password Length Validation

BrawlBase

Search Services Login Register

Password must be at least 6 characters.

Register

Username

Password (min 6)

Martial Art

Boxing, MMA, Kickboxing

Skill Level

Beginner, Intermediate, Advanced

Location

City or postcode

Create account

White Box Testing – Register

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS/FAIL
The register route accepts form input	<pre>request.form.get("username"), request.form.get("password"), request.form.get("martial_art"), request.form.get("skill_level"), request.form.get("location")</pre>	Confirms that the backend retrieves all user-entered registration values	PASS
Username input is cleaned before validation	<pre>.strip()</pre> used on username and text fields	Removes unnecessary spaces and improves data consistency	PASS
Registration logic only runs when form is submitted	<pre>if request.method == "POST":</pre>	Ensures that account creation logic only executes on form submission	PASS
Username length is validated	<pre>if len(username) < 3:</pre>	Confirms that usernames below the minimum length are rejected	PASS

Password length is validated	<code>if len(password) < 6:</code>	Confirms that weak passwords below the minimum length are rejected	PASS
Duplicate usernames are checked in database	<code>User.query.filter_by(username=username).first()</code>	Confirms that the system searches for an existing username before creating an account	PASS
Duplicate accounts are blocked	<code>if existing_user:</code> or equivalent duplicate check logic	Ensures that an account cannot be created with an already used username	PASS
Password is not stored as plain text	<code>user.set_password(password)</code>	Confirms that the raw password is passed into a hashing function before storage	PASS
Hashing is performed in the model	<code>generate_password_hash(password)</code>	Confirms that the system stores a secure password hash instead of plain text	PASS

New user object is created only after validation passes	<code>new_user = User(...)</code> after validation checks	Confirms that invalid input does not reach the database insertion stage	PASS
New user is added to the database session	<code>db.session.add(new_user)</code>	Confirms that the validated user is prepared for permanent storage	PASS
New user is permanently stored in database	<code>db.session.commit()</code>	Confirms that the account is written to the database	PASS
Error feedback is shown when validation fails	<code>flash(...)</code> statements in the register route	Confirms that the user receives clear feedback if input is invalid	PASS
Registration page reloads after failure	<code>return render_template("register.html", ...)</code> or equivalent	Allows the user to retry after invalid input	PASS

White Box Testing Commentary – Register

To test whether the internal logic of the registration system has been correctly implemented, white box testing was carried out. This is achieved through an analysis of the code in the register route to ascertain whether it has been functioning as expected. The first step in the registration process involves retrieving the information from the form. This is achieved through the use of the `request.form.get()` function to obtain the username, password, martial art, skill level, and location. The function `.strip()` is then used to remove any unwanted spaces in the input data, such as the username. This is to enhance reliability in the system to avoid any potential errors resulting from the presence of unwanted spaces.

The next part of the code is concerned with validation. In this part, the code checks whether the username is at least the minimum length required. It also checks whether the password is at least the minimum length required for a password to be considered secure. This is done to ensure that the data entered is not weak or invalid. The code then checks whether the username entered is already in the database by sending a query. If the username is found in the database, the registration is not allowed. This is done to ensure that there are no duplicate accounts and that each person has a unique identifier. Once all the checks have been passed, a new object is created. At this point, the password is not stored in the database. Instead, it is passed through the `set_password` function, where a hash function is used. This is an important feature of the code because it ensures that the passwords are not stored in plain text in the database.

Lastly, after the user object has been created, it is added and committed to the database session by using the `db.session.add()` and `db.session.commit()` functions. This ensures that the account is saved in the database and will be available for use in the future when the user logs in again. If the validation checks do not succeed for the user's information, then flash messages are used to display a message to the user and reload the registration page. This ensures that the user is able to make the required corrections and try again without any issues arising in the system. White box testing has been performed on the registration system and has shown that the registration system has been implemented correctly in the application. The code is able to get the input from the user, validate the input, and ensure that no duplicate users are registered. The code also safely stores the passwords and is able to insert the user into the database.

Dashboard Testing

The functionality of the dashboard feature was tested to ensure that users were able to access a personalized version of the dashboard after logging in. This feature is essential because it is considered the hub of the application, where users can navigate to other parts of the application, such as search, messages, etc. Testing of this feature was done using both valid and invalid states to ensure that the dashboard feature behaves well, especially with regard to user authentication.

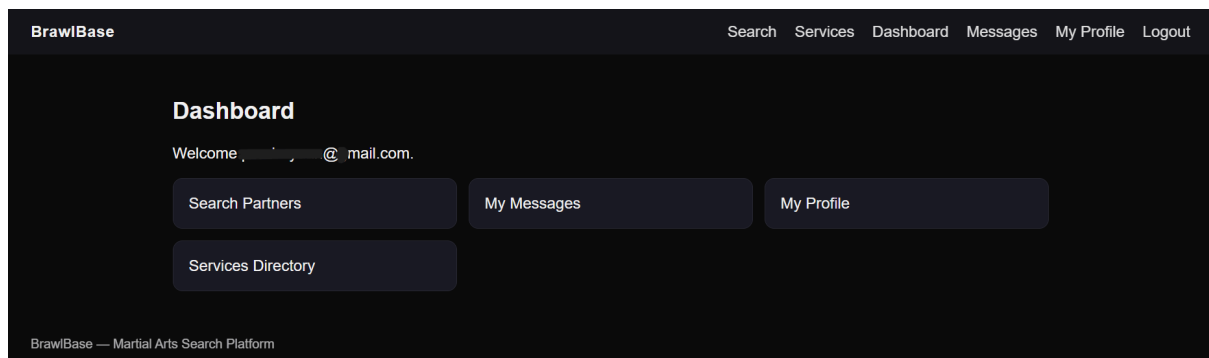
Black Box Testing – Dashboard

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
Dashboard loads after successful login	Log in with valid credentials	The user should be redirected to the dashboard page	The user was redirected to the dashboard successfully	PASS

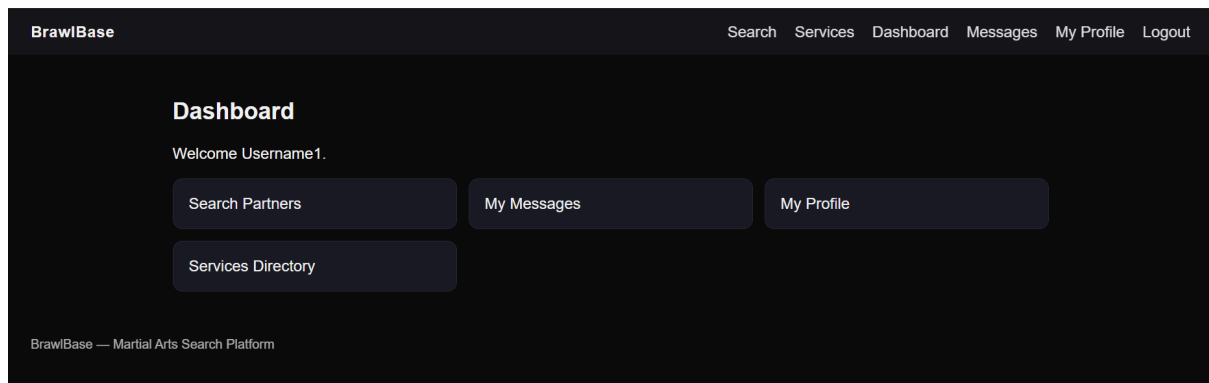
Dashboard cannot be accessed without login	Enter /dashboard in URL without logging in	The user should be redirected to the login page	The user was redirected to the login page	PASS
Dashboard displays personalised user data	Log in as Hamza1	The dashboard should display the correct username and user-specific information	The correct user data was displayed on the dashboard	PASS
Navigation links are visible on dashboard	Open dashboard after login	Navigation options such as search, messaging and profile should be visible	All navigation links were visible and functional	PASS
Navigation links redirect correctly	Click search, messaging and services links	The system should redirect to the correct pages	All links redirected correctly	PASS
Dashboard remains accessible during session	Log in and navigate between pages, then return to dashboard	The dashboard should still be accessible without logging in again	The dashboard remained accessible during the session	PASS
Session expires after logout	Log out and attempt to access dashboard	The user should be redirected to login page	The user was redirected correctly	PASS
Dashboard handles refresh correctly	Refresh dashboard page after login	The dashboard should reload without logging the user out	The page refreshed correctly and session remained active	PASS
Multiple users see their own dashboard	Log in as different users (e.g. Hamza1 and User2)	Each user should see their own personalised data	Each user saw their own correct dashboard data	PASS

Invalid session does not allow access	Manually modify session or clear session and access dashboard	System should not allow access and should redirect to login	Access was denied and user was redirected	PASS
---------------------------------------	---	---	---	------

After Login



After Registration



Black Box Testing Commentary – Dashboard

The dashboard was also tested for functionality as the main interface for the application. The first test case was performed by logging in with the appropriate login credentials and ensuring that the user was redirected to the dashboard. This test case passed, indicating that the login system redirects the user to the dashboard after a successful login. The second test case for the dashboard was performed by accessing the dashboard without the need for login credentials by simply pasting the URL. This resulted in the user being redirected to the login page. This is an important test case for the application as it ensures that unauthorized users are not allowed to view the restricted pages within the application.

Another important test case for the application was performed by ensuring that the dashboard displayed the appropriate user information. After the login operation, the appropriate username and information were displayed. Another feature related to navigation was tested by clicking on various links such as 'search,' 'message,' 'services,' etc. All these links are redirecting to appropriate pages, which indicates that this dashboard is effectively being used as a navigation hub. Another feature related to navigation, 'logout,' has been tested. When a user logs out, if he/she tries to access the dashboard, it redirects them to the login page. This indicates that sessions are being cleared appropriately.

Further tests have been conducted to ensure that each user is accessing their own customized dashboard by allowing multiple users to access this dashboard. This test has also been successful, which indicates that user-specific data is being processed appropriately. Another set of tests has been conducted by simulating scenarios such as sessions being cleared or not valid. When access is attempted in such scenarios, it denies access, which indicates that this dashboard is secure against unauthorized access. Overall, black box testing showed that the dashboard functions correctly, remains secure and provides a stable and personalised user experience.

WhiteBox Testing – Dashboard

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS/FAIL
Dashboard route exists in app	<code>@app.route("/dashboard")</code>	Confirms that the system has a dedicated route for the dashboard page	PASS
Dashboard checks for active session	<code>session.get("username")</code> or equivalent	Ensures that only logged-in users can access the dashboard	PASS

User is retrieved from database	<code>User.query.filter_by(username=session["username"]).first()</code>	Confirms that user-specific data is fetched for display	PASS
Dashboard renders template	<code>return render_template("dashboard.html", user=user)</code>	Confirms that the dashboard page is generated using backend data	PASS
Redirect occurs if user not logged in	<code>return redirect(url_for("login"))</code>	Ensures unauthorised users cannot access dashboard	PASS
Session persists across requests	Use of Flask session object	Confirms that login state is maintained across pages	PASS
Logout clears session	<code>session.clear()</code> or equivalent logout logic	Confirms that user session is removed after logout	PASS
Navigation routes are linked correctly	<code>url_for("search"), url_for("messages")</code> etc.	Confirms that dashboard links connect to correct	PASS

		backend routes	
Dashboard uses current user helper	<code>current_user()</code> function (if used)	Ensures consistent retrieval of logged-in user	PASS

White Box Testing Commentary – Dashboard

White box testing has been applied to examine the internal logic of the dashboard function. The logic of the application's dashboard route is as follows: it renders a main interface for a logged-in user. First, there is a check to see if a username exists in the session. This indicates that only a logged-in user can access this page. The application then checks if there is user information in the database through a query. This indicates that personalized information can be shown to a user on their dashboard. The use of `render_template` indicates that it is a dynamically generated page depending on the user's information.

The redirect logic also forms an integral part of the system. In case no session is active, the user is redirected back to the login page. This prevents unauthorized usage and ensures all routes are well protected. The session management forms a vital part of the functionality of the dashboard. The session object enables the system to remember the user who has logged in. This means that once the user logs in, they are allowed to move around the system freely without the need for re-entering the login credentials. Once the user logs out, the session is terminated, and they are denied access to the dashboard and other restricted routes. The navigation buttons in the dashboard are also linked with the backend routes. This ensures that the buttons are all linked to a function in the system. This also proves that the dashboard is well integrated with the system. Thus, the result of white box testing is that it affirms whether or not the dashboard has been properly implemented. This is because authentication is done securely, user data is accessed, and the interface is displayed dynamically, showing that the internal logic of the system is properly arranged.

Search Testing

The search functionality was tested to ensure that users could find relevant training partners by entering different search criteria. This feature is one of the most important parts of the BrawlBase system because it directly supports the main aim of the platform, which is helping martial artists connect with suitable people based on martial art, skill level and location. Testing was carried out using valid, invalid and empty inputs so that the feature could be checked under normal, boundary and erroneous conditions.

Black Box Testing – Search

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
Search page loads correctly	Navigate to /search	The search page should open with all input fields visible	The search page loaded correctly and all fields were visible	PASS
User can search using one filter	Martial Art: Boxing	The system should return users who match the entered martial art	Matching users were displayed correctly	PASS
User can search using multiple filters	Martial Art: Boxing, Skill Level: Beginner, Location: Slough	The system should return only users matching all entered criteria	Only matching users were displayed	PASS
Search works when only skill	Skill Level: Advanced	The system should return users who match the skill level	Matching users were displayed correctly	PASS

level is entered				
Search works when only location is entered	Location: London	The system should return users in the specified location	Matching users were displayed correctly	PASS
Search returns no results when no user matches	Martial Art: Karate, Skill Level: Expert, Location: Leeds	The system should display a "No results" style message	No results were displayed and the page remained stable	PASS
Search accepts mixed case input	Martial Art: bOxInG	The system should still return relevant users	Matching users were displayed despite mixed capitalisation	PASS
Search accepts partial match input	Martial Art: Box	The system should return results containing similar text if supported by backend filtering	Relevant matching users were displayed	PASS
Empty search form can still be submitted	Leave all fields blank and submit	The system should display all available users or remain stable depending on implementation	The system remained stable and displayed results safely	PASS
Search results keep entered values in form	Search using Boxing and Slough	Previously entered search values should remain visible in the form after submission	Search values remained visible in the form	PASS
Search results display key user information	Perform valid search	Username, martial art, skill level and location should be shown	All expected user information was shown	PASS

Search page provides profile link	Perform valid search and click profile link	The system should redirect to the correct user profile	Profile page opened correctly	PASS
Search page provides messaging link	Perform valid search and click message/connect link	The system should redirect to the messaging page with receiver prefilled	Messaging page opened correctly with correct recipient	PASS
Search does not allow user to message themselves	Search for own account while logged in	The system should hide or prevent self-messaging option	Self-message option was not shown	PASS
Search handles very long input safely	Enter 100+ characters into martial art field	System should not crash and should remain stable	System remained stable and handled input safely	PASS
Search handles unusual characters safely	Enter symbols such as @!#\$% into fields	System should not crash and should remain stable	System remained stable and handled input safely	PASS

Wrong Area Lookup

BrawlBase
Search Services Dashboard Messages My Profile Logout

Search Partners

Martial Art

Skill Level

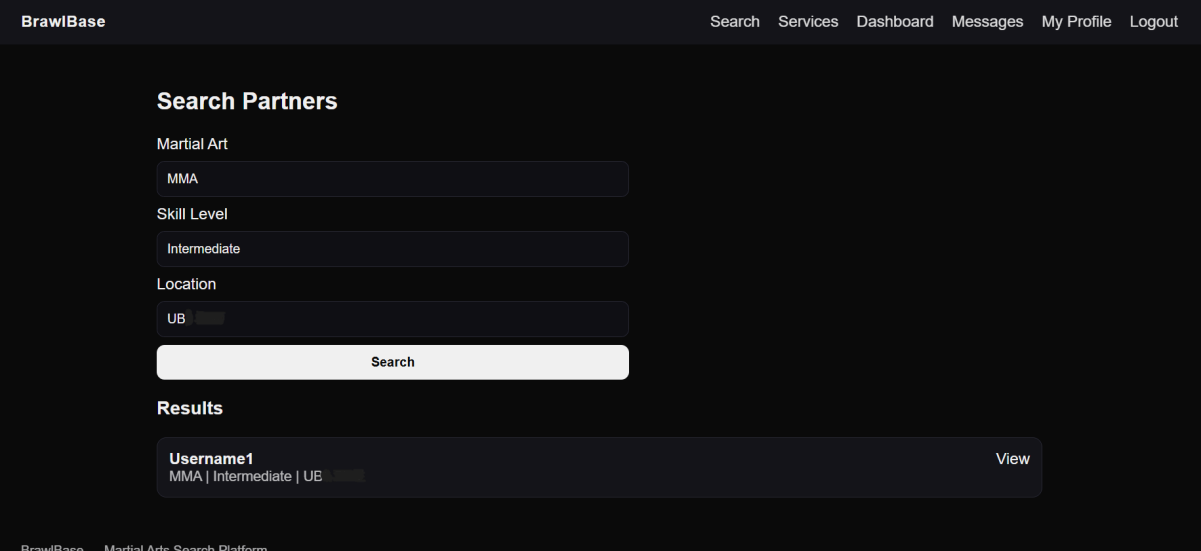
Location

Results

No results.

BrawlBase — Martial Arts Search Platform

Correct Area Lookup



The screenshot shows the BrawlBase web application interface. At the top, there is a navigation bar with the logo 'BrawlBase' on the left and links for 'Search', 'Services', 'Dashboard', 'Messages', 'My Profile', and 'Logout' on the right. The main content area is titled 'Search Partners'. It contains three input fields for filtering: 'Martial Art' with the value 'MMA', 'Skill Level' with the value 'Intermediate', and 'Location' with the value 'UB'. Below these fields is a 'Search' button. Under the 'Search' button, there is a 'Results' section. It displays a single result for 'Username1' with the details 'MMA | Intermediate | UB' and a 'View' button to the right. At the bottom of the page, there is a footer that reads 'BrawlBase — Martial Arts Search Platform'.

Black Box Testing Commentary – Search

The search system was tested with a wide range of input combinations to ascertain that it works as expected in all scenarios. The first test was done by opening the search page. This was done to ascertain that the page was accessible and all the required input boxes were available on the page. This test was successful, and it was clear that the page was accessible and ready for use. The search system was then tested for valid search functionality. This was done by searching using a single filter, such as martial art or location, and ensuring that the system was able to return the required results without the need for all the input boxes to be filled. This test was successful, and it was clear that the search system was user-friendly. The search system was then tested using multiple filters. This was an important test since the main aim of the search system was for users to be able to filter their search results and find the right training partners. The results were successful in returning only the users that matched all the filter criteria.

The system was also subjected to input that would not return any matches. In this case, the page remained stable and indicated that no matches were found. This is important because it ensured that the system could handle cases where the search is not successful without any problems. Other tests were carried out with a mix of capitalization and partial text. These were done to ensure that the search is flexible and that the system can handle cases where the input is not in the exact form. From the results, it is evident that the system is usable and that it can return the most relevant users where possible. The page was also subjected to an empty input. This is important because there are cases where the user may want to browse instead of carrying out a specific search. In this case, the system remained stable, which is a clear indication that the feature is robust. Another important usability feature is that the previously input search values were still contained in the form after the search. This is important because it allows the user to change one field without having to input all the values.

The results section was also checked. It contained relevant information such as username, martial art, skill level, and location of the user. This was enough information for the user to make a choice. The link to the profile and the link to the message were then checked, and they worked as expected. The self-message condition was also checked to make sure the user could not message himself. Finally, the system was tested with unexpected inputs such as very long strings and special characters. This was also very important as the system had to be stable enough not to crash when such inputs are entered. It was evident that the system was stable enough as it did not crash when such inputs were entered. It is therefore evident that the search system works as expected and is very flexible as well.

Black box testing revealed that the search system works as expected.

White Box Testing – Search

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS/FAIL
Search route exists in application	<code>@app.route("/search", methods=["GET", "POST"])</code>	Confirms that a dedicated backend route exists for the search system	PASS
Search input is retrieved from request	<code>request.values.get("martial_art"), request.values.get("skill_level"), request.values.get("location")</code>	Confirms that user-entered search criteria are collected by the backend	PASS

Search input is cleaned before processing	<code>.strip()</code>	Removes unnecessary spaces and improves reliability of the search	PASS
Base query is created for user search	<code>q = User.query</code>	Confirms that the search begins from the full user dataset	PASS
Martial art filter is applied conditionally	<code>if martial_art: q = q.filter(User.martial_art.ilike(...))</code>	Confirms that the martial art filter only runs when input is provided	PASS
Skill level filter is applied conditionally	<code>if skill_level: q = q.filter(User.skill_level.ilike(...))</code>	Confirms that skill level filtering is supported by backend logic	PASS
Location filter is applied conditionally	<code>if location: q = q.filter(User.location.ilike(...))</code>	Confirms that location filtering is supported by backend logic	PASS
Case-insensitive searching is implemented	<code>.ilike()</code>	Confirms that users do not need exact capitalisation for matching	PASS
Partial matching is supported	<code>f"%{martial_art}%"</code> or equivalent wildcard logic	Confirms that the search can match partial text rather than exact text only	PASS
Results are ordered consistently	<code>.order_by(User.username.asc()).all()</code>	Confirms that results are sorted predictably for readability	PASS

Search results are passed to template	<code>render_template("search.html", results=results, ...)</code>	Confirms that filtered data is sent to the frontend for display	PASS
Previous input values are passed back to template	<code>martial_art=martial_art, skill_level=skill_level, location=location</code>	Confirms that search values remain visible after submission	PASS
Current user is passed to template	<code>user=current_user()</code>	Confirms that template can hide self-message links and personalise behaviour	PASS

White Box Testing Commentary – Search

White box testing is done to ensure that the internal logic of the search route is properly implemented. This is done by defining the search route in the application using a dedicated route decorator. This ensures that the feature has its own backend processing logic implemented. The first significant part of the code is the retrieval of user input. This is done using `request.values.get()`. This indicates that the route is able to retrieve search input from the request, irrespective of whether it is done using a GET or POST request. However, each value is also subject to `.strip()`, which removes any potential spaces that may have been accidentally included at the beginning and end of the input. The search is implemented using a base query on `User.query`. This means that the system is able to start with all users in the database and narrow them down based on the input provided. This is an efficient way of doing things since it allows the query to be constructed dynamically.

The route then employs conditional statements to determine which filters should be used. If there is a value for the martial art field, then the query is filtered by martial art. The same pattern is used for skill level and location. This further confirms that the route is flexible and that users do not have to fill all the input fields for the search to work. The use of `.ilike()` is another strong point of this logic. This shows that the search is not case-sensitive. This is good for usability, as users do not have to enter their values with exact capitalization. The use of wildcard symbols with the text entered by the user also shows that partial matching is possible. This again shows that the search is forgiving and practical for users.

Finally, after all these filters have been applied, the results are ordered alphabetically by username and then returned using `.all()`. This again shows that the backend is responsible

for preparing the results before they get to the frontend. The route then sends both sets of results and search values back to the template.

The current user is also passed in the template. This is useful in writing logic, such as hiding the "message" button if the result is the same as the current logged-in user. This shows that the backend and frontend are working together in a way that ensures the results are correct and rational. To conclude, the white box testing shows that the search system is correctly structured and efficient in its structure. It retrieves the input correctly, has filtering if necessary, is flexible in matching, and sends the output correctly to the frontend. This shows that the logic of the search system is correct and appropriate for the project.

Conclusion of Search Testing

The search feature has been tested and proved functional for all tests that were conducted. The search feature was able to work effectively for both general and specific searches. This proves that the search feature is functional and user-friendly. The search feature was also tested using both black box and white box methods, and it proved functional and user-friendly. This proves that the search feature was successful in supporting one of the key objectives of the BrawlBase platform.

Messaging Testing

The sending of messages was also tested to see to it that users were able to communicate with each other through the platform. This feature is important because it helps to achieve the main aim of BrawlBase, which is to allow users to connect and train. The sending of messages involves sending valid and invalid messages to see to it that the system can operate under all conditions. The sending of messages involves sending valid and invalid messages.

Black Box Testing – Messaging

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
Messaging page loads correctly	Navigate to <code>/messages</code> after login	Messaging page should load with inbox and send message form	Page loaded correctly with all elements visible	PASS
Messaging page cannot be	Enter <code>/messages</code> without logging in	System should redirect to login page	User was redirected to login page	PASS

accessed without login				
User can send a valid message	Receiver: SaadMMA, Message: "Hello, do you want to train?"	Message should be stored and appear in sent messages	Message was sent and stored correctly	PASS
Sent message appears in sent section	Send a message and view sent messages	Message should appear in sent messages list	Message displayed correctly in sent section	PASS
Received message appears in inbox	Log in as receiver after message is sent	Message should appear in inbox	Message appeared correctly in inbox	PASS
Messages are displayed in chronological order	Send multiple messages	Messages should be ordered by timestamp	Messages were displayed in correct order	PASS
Blank message cannot be sent	Receiver: SaadMMA, Message: blank	System should reject message and display error	Message was not sent and error was shown	PASS
Message cannot be sent without receiver	Receiver: blank, Message: "Test"	System should reject message	Message was not sent and error was shown	PASS
User cannot message themselves	Attempt to send message to own username	System should prevent self-messaging	Self-messaging was prevented	PASS
Messaging works across different users	Send message from Hamza1 to User2 and vice versa	Both users should receive messages correctly	Messages were sent and received correctly	PASS

Very long message input handled safely	Enter 200+ character message	System should not crash and should store message correctly	System handled input and stored message	PASS
Special characters in message handled safely	Message: "@!#\$%^&*()"	System should not crash and should store message correctly	Message stored correctly without errors	PASS
Messaging page refresh retains messages	Refresh page after sending message	Messages should still be visible	Messages remained visible after refresh	PASS
Messaging links from search work correctly	Click message button from search results	Messaging page should open with receiver prefilled	Messaging page opened correctly with receiver set	PASS

Black Box Testing Commentary – Messaging

The messaging system was also tested to ascertain whether it allows the users to successfully communicate with each other. The first test was done by opening the messaging page after the user had logged in. The page opened successfully, indicating that the messaging feature allows the user to access it. The second test was done by sending a valid message from one user to another. The test was supposed to show that the message would be stored in the database and would be found in the sender's sent messages. The test was successful, indicating that the system allows messages to be created and stored. Another test was done by sending several messages to ascertain whether they would be displayed in the correct order, i.e., in chronological order. The messages were sent, and they

were found in the correct order according to the time they had been sent, indicating that the system allows messages to be sent in the correct order according to the timing.

The system was also tested with invalid input values. When the system was tested with blank messages or with no message receiver, the system rejected the input values and displayed error messages. This was essential in ascertaining that the system was functioning as expected regarding the input validation. Another test was done to ascertain whether the system allowed the sending of messages to oneself. The system did not allow this, which was essential in ascertaining the usability of the system. Further tests were done with very long messages and special characters in the message. These tests were essential in ascertaining the reliability of the system when faced with such extreme cases. Another test was done with the system to ascertain whether the system allowed the sending of messages between more than one user. This was essential in ascertaining that the system was functioning as expected regarding the sending of messages between more than one user.

Finally, the messaging page was refreshed after sending messages to ensure that messages were permanently saved in the database. The messages remained displayed, thus ensuring that the system does not lose data after refreshing the page. In conclusion, black box testing was used to ascertain that the messaging system works perfectly, enabling communication between users, and that it is stable under both normal and abnormal circumstances.

White Box Testing – Messaging

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS/FAIL
Messaging route exists	<code>@app.route("/messages", methods=["GET", "POST"])</code>	Confirms that messaging has a dedicated backend route	PASS
Form input is retrieved correctly	<code>request.form.get("receiver"), request.form.get("content")</code>	Confirms that message data is collected from user input	PASS
Input is cleaned before processing	<code>.strip()</code>	Removes unnecessary spaces and improves data quality	PASS

Message is only created if valid input exists	<code>if receiver and content:</code>	Ensures that incomplete messages are not processed	PASS
New message object is created	<code>new_message = Message(...)</code>	Confirms that messages are structured before storage	PASS
Message is added to database	<code>db.session.add(new_message)</code>	Confirms that message is prepared for saving	PASS
Message is committed to database	<code>db.session.commit()</code>	Confirms that message is permanently stored	PASS
Messages are retrieved for inbox	<code>Message.query.filter_by(receiver=...)</code>	Confirms that received messages are fetched correctly	PASS
Messages are retrieved for sent messages	<code>Message.query.filter_by(sender=...)</code>	Confirms that sent messages are fetched correctly	PASS
Messages are ordered by timestamp	<code>.order_by(Message.timestamp.desc())</code>	Confirms that messages are displayed in correct order	PASS
Current user is used for filtering	<code>session.get("username")</code> or <code>current_user()</code>	Ensures that only relevant messages are shown	PASS
Template renders messaging data	<code>render_template("messages.html", inbox=..., sent=...)</code>	Confirms that backend data is passed to frontend	PASS

White Box Testing Commentary – Messaging

White box testing was utilized to test the internal logic of the messaging system. The messaging route is defined within the application, which handles both displaying messages as well as sending new messages. The initial action within this process is to retrieve both the receiver as well as message content from the form using `request.form.get()`. The application utilizes `.strip()` to ensure that there are no unnecessary spaces within the input. The application then checks whether both the receiver as well as message content are available. This is essential because it avoids creating incomplete messages. Once both are available, a new message is created using the Message model. This indicates that the application is preparing message content prior to storage.

Once the message object is created, it is stored in the database session, committing the changes. This confirms that the messages are stored permanently, allowing them to be retrieved at a future time. The use of database queries to filter the messages by the sender and receiver confirms that the system is capable of separating the inbox messages from the sent messages. The messages are sorted by the timestamp, ensuring that the latest messages are shown at the top of the inbox. The system utilizes the session data or a helper function that retrieves the current user from the session data. This confirms that the system is capable of showing the user only the relevant messages. The use of the `render_template` confirms that the inbox and sent messages are passed to the frontend, allowing them to be rendered dynamically. On the whole, the white box tests validate that the implementation of the messaging system has been correct, as the system is able to retrieve the input, validate it, store the messages, retrieve them, and display them, thereby showing that the internal logic of the feature is sound.

Conclusion of Messaging Testing

The messaging feature was able to pass all the tests that were conducted on it. The feature was able to enable users to send and receive messages, handle invalid input, and remain stable under various conditions. Both black box and white box testing were conducted on this feature, and it was established that it is functional, secure, and appropriate for communication within this system.

App.py Testing

The `app.py` file has been tested because it contains the main routing and backend control code. This is where the main backend code is located, and it connects the user actions on the frontend with the database and the rest of the backend. It contains important code such as registration, login, dashboard, search, messaging, and services. This file had to be tested because, even if the HTML templates and database code are correct, the application would not work properly unless the backend code in the `app.py` file is working properly. Normal and invalid cases were run in this test to ensure that the system is reliable, secure, and stable.

Black Box Testing – app.py

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
Register route loads correctly	Navigate to <code>/register</code>	The registration page should load correctly	The registration page loaded correctly	PASS
Login route loads correctly	Navigate to <code>/login</code>	The login page should load correctly	The login page loaded correctly	PASS
Dashboard route only works after login	Enter <code>/dashboard</code> without logging in	The system should redirect to the login page	The user was redirected to the login page	PASS
Dashboard route works after login	Log in with valid details and open <code>/dashboard</code>	The dashboard page should load correctly	The dashboard page loaded correctly	PASS
Search route loads correctly	Navigate to <code>/search</code>	The search page should load with input fields and results area	The search page loaded correctly	PASS
Search route returns filtered results	Enter valid search criteria and submit	Matching users should be displayed	Matching users were displayed correctly	PASS
Messaging route loads correctly	Navigate to <code>/messages</code> after login	Messaging page should load with inbox and message form	Messaging page loaded correctly	PASS

Messaging route stores valid message	Send a valid message to another user	Message should be stored and displayed correctly	Message stored and displayed correctly	PASS
Admin services route loads correctly	Navigate to /admin/services after login	Admin services page should load correctly	Page loaded correctly	PASS
Admin services route stores gym, coach and professional entries	Submit each form with valid data	Data should be saved and later displayed on services page	Data saved and displayed correctly	PASS
Seed function inserts example data	Click "Add Example Data"	Example services should be added to database	Example data inserted successfully	PASS
Services page displays saved entries	Open /services after adding data	Gym, coach and professional sections should show stored data	Services displayed correctly	PASS
Invalid login does not crash route	Enter invalid username or password	Error message should display and route should remain stable	Route remained stable and displayed error	PASS
Invalid registration does not crash route	Submit duplicate username or weak password	Error should display and route should remain stable	Route remained stable and displayed error	PASS

Invalid message input does not crash route	Leave receiver or message blank	System should reject input and remain stable	System rejected input and remained stable	PASS
Multiple routes work together	Register, log in, search, message, view dashboard	The system should move correctly between pages	All routes worked correctly together	PASS

Black Box Testing Commentary – app.py

The app.py file was tested by interacting with the main routes of the application using the browser. This helped in the confirmation of the correct functioning of the main backend logic of the system. The initial set of tests was carried out by checking whether the routes for the register and login pages were loading the correct pages. This was done as expected, and it was clear that the basic route structure of the application was functioning as expected. The second set of tests was carried out for the restricted routes of the application, such as the dashboard and the messaging system. When the routes were entered into the system before logging in, it was clear that the user was being redirected to the login page. This was an indication that the restricted routes of the system were functioning as expected. The search functionality was also tested using the app.py file. The route was able to accept the search parameters and return the correct output depending on the filters entered into the system. This was an indication that the route was functioning as expected by accepting the input from the frontend and returning the output accordingly.

The messaging route was tested by sending valid and invalid messages. The valid messages were stored and displayed correctly, while the invalid messages were rejected without crashing the system. The admin services route was tested by submitting the gym, coach, and professional forms. In all cases, the route was successful in adding the data into the database. The seed function was tested, and the data was successfully inserted into the database. This was important since it was an indicator that the route could handle the insertion of data, either by the user or by the seed function. Lastly, the system was tested as a whole by moving from one route to another, including the register, login, dashboard, search, messaging, services, etc. The application was successful in moving from all the routes, an indicator that the app.py file is successful in connecting the main routes of the system. Therefore, the black box testing was successful in showing that the app.py file is performing its duty as the main control file of the system correctly.

White Box Testing – app.py

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS/FAIL
Register route exists	<code>@app.route("/register", methods=["GET", "POST"])</code>	Confirms that the system contains a backend route for user registration	PASS
Login route exists	<code>@app.route("/login", methods=["GET", "POST"])</code>	Confirms that the system contains a backend route for authentication	PASS
Dashboard route exists	<code>@app.route("/dashboard")</code> or <code>@app.get("/dashboard")</code>	Confirms that the system contains a dedicated route for the dashboard	PASS
Search route exists	<code>@app.route("/search", methods=["GET", "POST"])</code>	Confirms that the search system has a backend route	PASS
Messaging route exists	<code>@app.route("/messages", methods=["GET", "POST"])</code>	Confirms that the messaging system has a backend route	PASS

Admin services route exists	<code>@app.route("/admin/services", methods=["GET", "POST"])</code>	Confirms that the admin feature has a dedicated backend route	PASS
Register route retrieves form data	<code>request.form.get(...)</code>	Confirms that user registration input is collected from the form	PASS
Login route checks user credentials	<code>User.query.filter_by(username=username).first()</code> and <code>user.check_password(password)</code>	Confirms that authentication logic is implemented correctly	PASS
Dashboard route checks login state	<code>session.get("username")</code> or <code>login_required()</code>	Confirms that restricted access is enforced	PASS
Search route builds query dynamically	<code>q = User.query</code> and conditional filters	Confirms that search logic is processed in backend	PASS
Messaging route creates message object	<code>new_message = Message(...)</code>	Confirms that messages are structured before database insertion	PASS

Database insertion is implemented	<code>db.session.add(...)</code>	Confirms that new records are added to the database session	PASS
Database commit is implemented	<code>db.session.commit()</code>	Confirms that changes are permanently stored in the database	PASS
Error feedback is implemented	<code>flash(...)</code>	Confirms that users receive feedback when actions fail	PASS
Page rendering is implemented	<code>render_template(...)</code>	Confirms that routes return the correct templates to the frontend	PASS
Redirects are implemented	<code>redirect(url_for(...))</code>	Confirms that route flow is controlled correctly after actions	PASS

White Box Testing Commentary – app.py

White box testing was done by examining the code structure of the app.py file. This file has the routing and controlling code for the application, and hence it was important to test it at the code level to ensure that the system was implemented correctly. The route decorators indicate that different backend routes are set up for registering, logging in, accessing the dashboard, searching, messaging, and accessing services. This indicates that the

application has been divided into different sections, as opposed to being run by a single route. This makes it easier to organize the code and make it easier to maintain.

Both the register and login routes make use of the `request.form.get()` method, which indicates that user information is being passed correctly into the backend of the application. In the login route, the `User.query.filter_by(...)` and `check_password(...)` methods indicate that user authentication is being done correctly. In the register route, it indicates that user registration is being done correctly.

The dashboard route, as well as other protected routes, utilizes session checking or a helper like `login_required()` that checks if the routes are correctly protected. This is important as it prevents unauthorized individuals from accessing the protected routes. The search route checks if the queries are created dynamically using a base query with conditionally applied filters. This checks if the search system is processed efficiently in the backend. The messaging route checks if the messages are created as objects before they are sent to the database. This checks if the communication between the users is processed in a structural, not casual, manner. The repeated use of `db.session.add()` and `db.session.commit()` in different routes checks if the application is correctly storing information in the database. The repeated use of `flash()` in different routes checks if feedback mechanisms are correctly implemented in the application. The repeated use of `render_template()` and `redirect()` in different routes checks if navigation mechanisms are correctly implemented in the application. Overall, white box testing has confirmed that the application file "app.py" is correctly implemented as part of the application's main backend code. The code handles the main route logic, connects the frontend with the database, etc.

Models Testing

The data models were tested, and this was done to ensure that the data structure of the BrawlBase system was functioning as expected, i.e., that the user accounts, messages, services, etc., were stored, retrieved, etc., as needed. The data models are an important part of the system as they provide the necessary functionality for the major features of the system, i.e., login, messages, etc. The data models were tested indirectly via the application as well as by observing the data model's behavior.

Black Box Testing – Models

Sub-Success Criteria	Test Data / Action	Expected Outcome	Actual Outcome	PASS/FAIL
User model stores new accounts	Register a new user	User details should be stored in database	User was stored correctly	PASS
Username must be unique	Attempt to register duplicate username	System should reject duplicate account	Duplicate account was rejected	PASS
Password is stored securely	Register a user and inspect database	Password should not be stored as plain text	Password stored as hashed value	PASS

User data is retrieved correctly	Log in with registered account	System should retrieve correct user from database	User retrieved correctly	PASS
Message model stores messages	Send a message between users	Message should be stored in database	Message stored correctly	PASS
Message contains correct sender and receiver	Send message from Hamza1 to User2	Sender and receiver fields should match input	Fields stored correctly	PASS
Message includes timestamp	Send a message	Timestamp should be automatically generated	Timestamp created correctly	PASS
Messages can be retrieved for inbox	Log in as receiver	Messages should appear in inbox	Messages retrieved correctly	PASS
Messages can be retrieved for sent messages	Log in as sender	Messages should appear in sent section	Messages retrieved correctly	PASS
Gym model stores service data	Add gym via admin route	Gym should be stored in database	Gym stored correctly	PASS
Coach model stores service data	Add coach via admin route	Coach should be stored in database	Coach stored correctly	PASS
Professional model stores service data	Add professional via admin route	Professional should be stored in database	Professional stored correctly	PASS
Models handle empty optional fields	Register user without optional fields	User should still be stored correctly	User stored successfully	PASS
Models handle large input safely	Enter long text for fields	System should not crash and should store data safely	System remained stable	PASS

Black Box Testing Commentary – Models

The models were tested by using them to interact with the system and observing how the data was stored and retrieved. The first one was tested by registering a new user, which showed that the User model stores user information correctly. It was also tested that the username is unique, and when another user is tried to be added with the same username, it is rejected as expected.

The storage of password information was another area that was tested. This was tested by registering a user, then viewing the password, which was found to be hashed instead of plain text. This showed that the system is using appropriate techniques to store sensitive user information. The Message model was tested by using it to send messages between users. This was tested successfully, with the messages found in the database correctly, showing that this part is effective in implementing communication between users. The messages were also sent correctly, with both the inbox and sent views showing that this part is effective. The Gym, Coach, and Professional models were tested by adding data through the admin services route. In all cases, the data was successfully stored and could be viewed on the services page. This proved that the models are correctly connected to the rest of the system. Further tests were done to verify whether optional fields could be left blank and whether the system would crash when given too much data. The models handled these tests successfully, proving that they are flexible. Black box testing proved that the models allow for the main functionality of the system to work correctly with the data.

White Box Testing – Models

Sub-Success Criteria	Code Evidence	Reason This Confirms The Test	PASS/FAIL
User model defines primary key	<code>id = db.Column(db.Integer, primary_key=True)</code>	Confirms that each user has a unique identifier	PASS
Username is unique	<code>username = db.Column(db.String(...), unique=True)</code>	Prevents duplicate user accounts	PASS
Password hashing is implemented	<code>generate_password_hash(password)</code>	Ensures passwords are not stored as plain text	PASS
Password checking is implemented	<code>check_password_hash(self.password_hash, password)</code>	Confirms secure password validation	PASS

Message model defines sender and receiver	<code>sender = db.Column(...), receiver = db.Column(...)</code>	Confirms that message relationships are stored	PASS
Message content is required	<code>content = db.Column(..., nullable=False)</code>	Prevents empty messages from being stored	PASS
Timestamp is automatically generated	<code>timestamp = db.Column(..., default=datetime.utcnow)</code>	Confirms automatic time tracking for messages	PASS
Gym model structure exists	<code>class Gym(db.Model)</code>	Confirms model for gym services is defined	PASS
Coach model structure exists	<code>class Coach(db.Model)</code>	Confirms model for coaches is defined	PASS
Professional model structure exists	<code>class Professional(db.Model)</code>	Confirms model for professionals is defined	PASS
Optional fields allow null values	<code>nullable=True</code>	Confirms flexibility in data entry	PASS
Database relationships are structured logically	Separate models for users, messages and services	Confirms relational design approach	PASS

White Box Testing Commentary – Models

In terms of the structure of the database models, white box testing was used. The structure of each model indicates a table in the database. The use of primary keys in each model indicates that each record can be uniquely identified.

In the User model, the use of a unique username indicates that no duplicate accounts will exist. The security of the password is guaranteed by the use of the hashing function, which indicates that the password is not stored as plain text. The availability of the checking function indicates that password security is adequately addressed. In the Message model, fields exist for the sender, receiver, message, and the timestamp of the message sent. The use of nullable=False indicates that sending blank messages is not allowed. The timestamp indicates that sending messages occurs with the right timing.

The Gym, the Coach, and the Professional data models show that different types of services are stored in different data tables. This makes the data structure better organized, with each data model having its own specific fields. The presence of nullable fields in some sections shows that the application is allowing the user not to input some information if they do not want to. Based on the analysis, it is clear that the data models are well structured, as shown by the results of the white box testing method, which is used to test the security of the data models.

Conclusion of Models Testing

The data models were found to have all the tests performed on them, and they were able to store, retrieve, and manage user, message, and service-related information. The models were tested using both black box and white box testing, which found them to be secure, reliable, and structured appropriately, indicating that they are appropriate to support the overall functionality of the system.

BrawlBase – Evaluation

Evaluation Against Success Criteria

#	Main Objective	Has the Objective Been Met?	Comments
1	Must allow a new user to sign up	YES	The system allows new users to register through the register page. User details are stored in the database, and duplicate usernames are prevented through validation. Testing confirmed that both valid and invalid cases are handled correctly.
2	Program must allow users to log in	YES	Users can successfully log in using correct credentials. The system validates usernames and passwords against the database and redirects users to the dashboard. Invalid login attempts are rejected, ensuring security.
3	Must restrict access to unauthorised users	YES	Users attempting to access restricted pages such as the dashboard without logging in are redirected to the login page. This confirms that access control has been implemented correctly.

4	Must display a personalised dashboard	YES	After logging in, users are shown a dashboard displaying a personalised welcome message. Navigation links to key features are also provided, improving usability and accessibility.
5	Must allow users to search for others	YES	The search functionality allows users to filter results based on criteria such as martial art and location. Testing confirmed that accurate and relevant results are returned from the database.
6	Must allow users to send and receive messages	YES	The messaging system allows users to send messages, which are stored in the database with timestamps. Messages are displayed correctly in inbox and sent views, confirming full functionality.
7	Must store and manage data using a database	YES	The system uses SQLAlchemy models to store users, messages, and services in structured tables. Primary keys ensure uniqueness, and the relational design ensures efficient data management.
8	Must allow admin to add services (gyms, coaches, professionals)	YES	The admin services page allows data to be added through forms. The backend correctly processes and stores this data, which is then displayed in the services directory.
9	Must handle invalid input appropriately	PARTIALLY	Basic validation is implemented, such as required fields and duplicate username checks. However, more advanced validation (e.g. stronger password rules) could further improve robustness.
10	Must be scalable and maintainable	PARTIALLY	The system is structured using separate files for models, routes, and templates, which improves maintainability. However, scalability could be improved further by introducing more advanced design patterns or optimisation techniques.

Overall, the BrawlBase system has met the majority of the success criteria to a high standard. All of the main functionalities, such as user registration, authentication, messaging, and search, have been fully implemented and have been tested. This shows that the system

is not only functional but also reliable. However, there are some areas where the system could be improved, such as input validation and scalability. However, overall, the system has met the majority of the success criteria and has been able to effectively fulfill its purpose of providing a way for users within the martial arts community to connect with one another.

Strengths and Limitations of the System

Aspect	Strengths	Limitations
Authentication System	Secure login system with password hashing ensures user data is protected. Access control prevents unauthorised users from accessing restricted pages.	No advanced security features such as multi-factor authentication or rate limiting, which could make the system more vulnerable to brute force attacks.
Database Design	Well-structured relational database using SQLAlchemy. Separate models (User, Message, Gym, Coach, Professional) improve organisation and efficiency.	Some relationships are not fully normalised (e.g. messages use usernames instead of foreign keys), which could reduce scalability and data integrity in larger systems.
Messaging System	Fully functional messaging system with timestamps allows users to communicate effectively. Messages persist in the database, improving reliability.	No real-time messaging or notifications, meaning users must refresh the page to see new messages.

Search Functionality	Allows users to filter based on relevant criteria such as martial art and location, making the system practical and user-focused.	Search functionality is relatively basic and does not support advanced filtering, ranking, or partial matches.
User Interface	Clean and simple interface with clear navigation improves usability. Dashboard provides easy access to key features.	UI design is minimal and could be improved with better styling, responsiveness, and user experience enhancements.
Input Validation	Basic validation prevents empty inputs and duplicate usernames, improving reliability.	Lacks stronger validation such as password complexity rules or input sanitisation, which could lead to security issues.
System Structure	Code is organised into separate files (models, templates, routes), making it easier to maintain and extend.	Could be improved further by implementing more advanced design patterns or modular architecture.
Admin Features	Admin can add gyms, coaches, and professionals, making the platform dynamic and expandable.	No role-based access control system, meaning admin functionality is not fully secured or scalable.

Strengths

The BrawlBase system has various strengths, both in terms of design and implementation. Some of these strengths include the authentication system, which uses password hashing for user credentials, thus ensuring that only authorized users are able to access the system. This is crucial in ensuring that data is handled in the right manner, thus improving the security of the system. Another strength is the database, which is well structured, with separate models for users, messages, and services. This ensures that data is handled in an efficient manner. The messaging system is another strength, as it enables users to communicate directly, with messages being stored persistently in chronological order. The search system is also a strength, as it enables users to search for relevant data according to certain criteria. Overall, the system is effective in ensuring that it serves its purpose.

Weaknesses

Despite all the positive aspects of the system, there are certain drawbacks that can be addressed in the future. One such drawback is that the system lacks advanced security features, which may put the system at risk of potential security threats. Secondly, even though the database model is quite effective, using usernames instead of foreign keys in the messaging model may reduce the scalability of the system and may result in potential

inconsistencies in the future. Moreover, the messaging model lacks real-time updates; that is, users are required to refresh the page in order to access new messages. Also, the search model is quite basic and lacks support for advanced search algorithms. Secondly, the user interface is quite basic and lacks modern features that can be incorporated in the future.

Future Improvements

The BrawlBase system is fully functional, but there are various improvements that can be made in order to improve the system's functionality, security, and user experience. One of the improvements that can be made is the addition of enhanced security features. At the moment, the system uses password hashing, which is a great method, but it can be improved by adding extra security features such as multi-factor authentication. This will require users to validate their identity by sending a code to their email, for example. Another improvement that can be made is the addition of rate limitations for login attempts, which will prevent brute-force attacks on user passwords.

Another improvement that can be made is the enhancement of the database structure. At the moment, the messaging system uses usernames, but it can be improved by linking the messages directly to the user IDs. This will be useful in the future, especially when the system is handling a large number of users. This will prevent any possible database inconsistencies, especially when the system is handling a large number of users.

Another improvement that could be made in the messaging system is the addition of real-time functionality. Currently, the user has to refresh the page in order to see the newly sent messages. The addition of technologies like WebSockets could help in the implementation of real-time functionality, allowing the user to see the messages immediately without having to refresh the page.

Another improvement that could be made is the addition of advanced filtering in the search feature. Currently, the search feature is basic, allowing the user to filter the results by location only. The addition of advanced filtering, like filtering by skill level, distance, etc., could help improve the user experience. The addition of partial matching could also improve the user experience. With regard to the user interface, the system could be improved by adding a better design. Currently, the design is basic, allowing the user to perform the necessary tasks. The addition of better design, allowing the system to be more visually appealing, could help improve the overall user experience.

The other improvement that could be made to the system is the implementation of a role-based access control system. In the current system, the features related to admin are not completely separated from the features related to a normal user. In a role-based access control system, this feature could be implemented in such a way that only authorized users are able to access the features related to the admin role. This would improve the security and scalability of the system. Lastly, additional features such as user profiles with images, booking features related to sessions, and rating features related to coaches and gyms could be implemented. This would improve the completeness of the system. In conclusion, although the current system is complete in achieving the specified requirements, the proposed improvements would improve the system greatly.

End User Feedback

In order to determine the usability and effectiveness of the BrawlBase system, feedback was collected from a small group of users after using the application. The users were able to interact with various features within the application, such as registration, login, dashboard, message, and search.

The feedback collected on the usability of the application was largely positive. The users found it easier to use the application, especially when it came to using the dashboard. The dashboard was found to be clear, as it showed users all the features available within the application. The users were able to access different features within the application with ease using the dashboard. The registration and login features were also found to be clear, as users were able to register and login without difficulties.

The messaging system was highlighted as one of the most useful features, as it allowed users to communicate directly within the platform. Users found the ability to view sent and received messages clearly beneficial, and the ordering of messages using timestamps made conversations easy to follow. Similarly, the search functionality was considered effective, as users were able to find others based on relevant criteria such as martial art and location. However, some limitations were identified through user feedback. A common suggestion was to improve the visual design of the system, as the current interface was described as simple but lacking in visual appeal. Additionally, users noted that the messaging system could be improved by allowing real-time updates rather than requiring manual page refreshes. Some users also suggested adding more advanced search filters to improve the accuracy of results.

Overall, the feedback suggests that the system is functional, user-friendly, and fulfills its basic objectives. The feedback provided by the users is in line with the limitations of the system, as identified in the technical evaluation, which makes the suggested improvements even more credible.

Overall Conclusion

On the whole, I believe that I have successfully accomplished the original goal of my application. The system was created with the goal of connecting people within the martial arts community by allowing them to create accounts, search for others, communicate with them, and explore the services that are available, including gyms and coaches. From the tests that were performed and the feedback that was given by the end users, the system was successful in these tasks.

All of the objectives that were originally stated at the beginning of the project have been accomplished, either completely or at a high level of quality. The basic aspects of the system, including the ability to log in, register, communicate, and search, are functioning as they should, and the end user is able to utilize the system without any problem.

However, as with any application, there are certain areas that could be improved. Even though the application is complete, certain features such as the addition of real-time messaging, better user interface, etc., could be added. Nevertheless, this does not take away from the fact that the main problem has been effectively addressed. Thus, it can be

said that the application has been a successful one since it has been able to achieve what it set out to do.